DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

[INSTITUTION / UNIVERSITY NAME]

A MAJOR PROJECT STAGE-I REPORT ON

WORKUP: VIRTUAL PROFESSIONAL EXPERIENCE PLATFORM

A Containerized Simulation Environment for Situated Software Engineering Education, Authentic Team Workflows, and Cryptographically Verifiable Skill Demonstration

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY:

[Student Name]

Roll No: [Roll Number / USN]

UNDER THE GUIDANCE OF:

[Supervisor Name]

[Supervisor Designation]

Dept. of Computer Science & Engineering

ACADEMIC YEAR 2026–2027

CERTIFICATE

This is to certify that the Major Project Stage-I Report entitled "WORKUP: Virtual Professional Experience Platform" is a bonafide record of technical work carried out by [Student Name] (Roll Number: [Roll Number / USN]), in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from [Institution / University Name] during the academic year 2026–2027.

The results embodied in this report have been verified through actual codebase inspection, architectural modeling, and automated test execution. The work has not been submitted to any other University or Institution for the award of any degree or diploma.

[Supervisor Name]	**[Head of Department Name]**
Internal Guide & Supervisor	**Head of Department**
Department of CSE	**Department of CSE**

CANDIDATE DECLARATION

I, [Student Name], student of Bachelor of Technology in the Department of Computer Science and Engineering at [Institution / University Name], hereby declare that the work presented in this Major Project Stage-I Report entitled "WORKUP: Virtual Professional Experience Platform" is an authentic record of our own research, system design, architectural modeling, and prototype implementation.

I confirm that the facts, database models, testing results, and system flows described in this report strictly reflect the verified source code repository and original architectural specifications. I have adhered to all academic integrity guidelines and have fully cited all external literature, tools, and platforms consulted.

_____

[Student Name]

Roll No: [Roll Number / USN]

B.Tech (CSE), Final Year

ACKNOWLEDGEMENTS

I express my profound gratitude and indebtedness to my project guide and supervisor, [Supervisor Name], for their invaluable guidance, constant motivation, insightful suggestions, and technical critiques throughout the formulation, architecture design, and initial implementation phases of this Major Project.

I extend my sincere thanks to [Head of Department Name], Head of the Department of Computer Science and Engineering, for providing the necessary computing facilities, laboratory resources, and academic environment conducive to advanced technical research.

I also convey my deep appreciation to all faculty members, technical staff, and project coordinators of the Department of Computer Science and Engineering for their continuous encouragement, timely reviews, and constructive feedback during departmental project evaluations.

Finally, I am profoundly grateful to my family and peers for their unfailing encouragement, patience, and support throughout the course of this engineering endeavor.

ABSTRACT

A pronounced gap currently exists between theoretical computer science education and practical corporate software engineering. While conventional learning platforms (such as MOOCs and competitive programming websites) successfully convey isolated algorithmic concepts and syntax, they leave learners ill-prepared for corporate engineering realities. Newly graduated engineers frequently struggle when confronted with pre-existing multi-file legacy repositories, Architecture Decision Records (ADRs), non-trivial technical debt, multi-stage CI/CD pipelines, line-level pull request (PR) reviews, and live production telemetry alerts.

To overcome these systemic shortcomings, this project introduces WORKUP — a production-grade Virtual Professional Experience Platform. Grounded in David Kolb's Experiential Learning Theory and Situated Cognition, WORKUP transitions students from passive content consumers to active practicing engineers inside simulated corporate workplaces. Instead of solving toy programming problems, learners select engineering career tracks (spanning Software Engineering, MLOps, Cloud/DevOps, and Cybersecurity), assume specialized roles, and enter simulated tech enterprises (e.g., FinFlow Technologies, OmniRetail). They receive structured business problem tickets, open containerized cloud workspaces, inspect legacy codebases, implement features adhering to architectural constraints, execute automated test harnesses, submit pull requests, and receive automated, line-anchored feedback from an AI Tech Lead reviewer.

In this Major Project Stage-I phase, the platform's foundational architecture has been researched, designed, implemented, and empirically validated. The system is built upon Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, Prisma ORM with a 21-entity relational domain schema, bcryptjs/jose authentication, a 4-tier Role-Based Access Control (RBAC) matrix, a Directed Acyclic Graph (DAG) task engine, and an isolated sandbox execution orchestrator. An automated test suite powered by Vitest achieves a 100% pass rate across 34 tests in 8 suites, validating process jailing, multi-tenant isolation, and schema integrity. WORKUP culminates in an immutable, cryptographically verifiable Evidence-of-Skill Dossier backed by real Git commits, providing recruiters and academic evaluators with tamper-evident proof of practical competence.

TABLE OF CONTENTS

Certificate	ii
Candidate Declaration	iii
Acknowledgements	iv
Abstract	v
Table of Contents	vi
List of Figures	ix
List of Tables	x
List of Abbreviations	xi
CHAPTER 1: INTRODUCTION	13
1.1 Background and Motivation	13
1.2 Project Overview: WORKUP Platform	14
1.3 Problem Formulation and Problem Statement	14
1.4 Objectives of the Project	15
1.5 Scope of the Project	16
1.6 Existing Systems and Approaches	17
1.7 Limitations of Existing Approaches	17
1.8 Proposed WORKUP System and Key Innovations	18
1.9 Expected Benefits and Pedagogical Impact	18
1.10 Feasibility, Assumptions, and Technical Constraints	19
CHAPTER 2: LITERATURE SURVEY	20
2.1 Introduction to the Research Domain	20
2.2 Review of Relevant Academic Research Papers	20
2.3 Review of Existing Commercial and Industrial Platforms	23
2.4 Comprehensive Comparative Analysis Matrix	24
2.5 Identified Research and Architectural Gaps	24
2.6 System Justification and Formulation	25
CHAPTER 3: REQUIREMENTS AND METHODOLOGY	26
3.1 System Overview	26
3.2 Target Users and Role Personas	26
3.3 Functional Requirements Specification (FR-01 to FR-12)	27
3.4 Non-Functional Requirements Specification (NFR-01 to NFR-08)	28
3.5 Hardware and Software Specifications	29

3.6 Technology Stack and Justification	30
3.7 Software Development Methodology (Iterative Agile)	31
3.8 Main Operational Workflows	31
3.9 Data Management, Privacy, and Security Controls	32
3.10 Feasibility Analysis and Technical Constraints	32
CHAPTER 4: SYSTEM DESIGN AND UML MODELING	33
4.1 System Architecture & Tiered Topology	33
4.2 Use Case Modeling	34
4.3 Activity Modeling	36
4.4 Sequence Modeling (Task Submission & Review)	37
4.5 Class and Domain Object Modeling	38
4.6 Entity-Relationship (ER) Modeling	40
4.7 Component Modeling	40
4.8 Deployment Modeling	41
4.9 Universal Professional Lifecycle Pipeline	42
CHAPTER 5: MODULES AND FUNCTIONAL DESCRIPTION	44
5.1 Authentication, Session Management, and RBAC Module	44
5.2 Career, Profession, and Role Explorer Module	44
5.3 Simulated Enterprise Companies, Teams, and Projects Module	44
5.4 Simulation Content Engine, Task Backlog, and DAG Module	45
5.5 Student Workstation, Workbench UI, and Profile Module	45
5.6 Cloud Workspace Orchestration and Sandbox Provider Module	45
5.7 Task Implementation, In-Sandbox Test Execution & Snapshot	46
5.8 Multi-Tier Evaluation, Rubrics & AI Tech Lead Reviewer	46
5.9 Cryptographically Verifiable Portfolio & Evidence-of-Skill	46
5.10 Content Authoring Studio and Administrative Governance	46
5.11 Platform Auditing, Telemetry, and Fleet Monitoring	47
CHAPTER 6: IMPLEMENTATION PLAN AND	48

EXPECTED OUTCOMES

6.1 Current Implementation Status (Verified Codebase Facts) 48

6.2 Phased Milestone Roadmap (Phases 0 through 12) 48

6.3 Testing and Validation Approach (Vitest Empirical Results) 49

6.4 Expected Quantitative and Qualitative Outcomes 50

6.5 Risk Management, Threat Modeling, and Mitigation 51

6.6 Current System Limitations and Technical Debt 52

6.7 Proposed Stage-II Enhancements 52

CONCLUSION AND FUTURE WORK 53

REFERENCES 54

APPENDICES 56

Appendix A: Complete Database Schema Data Dictionary 56

Appendix B: Core RESTful API Endpoint Specifications 57

Appendix C: Sample Architectural Decision Record (ADR-001) 57

LIST OF FIGURES

Figure 4.1	WORKUP Tiered System Architecture Diagram (Client, API, Services, Execution, Storage)	33
Figure 4.2	WORKUP Unified Use Case Diagram (Student, Content Author, Reviewer, Administrator)	35
Figure 4.3	WORKUP Activity Workflow Diagram (Universal Lifecycle, Branching, Decision Points)	36
Figure 4.4	WORKUP Sequence Diagram for Task Submission, Sandbox Testing, and AI PR Review	37
Figure 4.5	WORKUP Core Domain Class Model Diagram (21 Entities, Attributes, Methods, Multiplicity)	39
Figure 4.6	WORKUP Entity-Relationship (ER) Relational Schema Diagram	40
Figure 4.7	WORKUP Component Architecture Diagram (App Router, API Handlers, Core Libs, Storage)	41
Figure 4.8	WORKUP Deployment Topology Diagram (Browser, Next.js Server, Sandbox Host, Persistent DB)	42
Figure 4.9	The Universal Professional Engineering Lifecycle Pipeline (14-Stage Connected Workflow)	43

LIST OF TABLES

Table 2.1	Comprehensive Comparative Analysis of Existing Platforms vs. Proposed WORKUP Platform	24
Table 3.1	Detailed Functional Requirements Specification Matrix (FR-01 to FR-12)	27
Table 3.2	Non-Functional Requirements and System Quality Attributes (NFR-01 to NFR-08)	28
Table 3.3	Hardware and Software Environment Specifications	29
Table 3.4	Technology Stack Components and Engineering Justification Matrix	30
Table 6.1	Phased Milestone Implementation Roadmap (Phases 0 through 12)	48
Table 6.2	Vitest Automated Test Suite Execution Summary (34 Tests, 8 Suites, 100% Passing)	49
Table 6.3	Risk Matrix, Severity Classification, and Engineering Mitigation Strategies	51
Table A.1	Complete Database Entity Dictionary and Relationship Mapping (21 Prisma Models)	56

LIST OF ABBREVIATIONS

ADR	Architecture Decision Record
API	Application Programming Interface
AST	Abstract Syntax Tree
CI/CD	Continuous Integration and Continuous Deployment
CLI	Command Line Interface
CPU	Central Processing Unit
DAG	Directed Acyclic Graph
DBMS	Database Management System
DOM	Document Object Model
DSA	Data Structures and Algorithms
ER	Entity-Relationship
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
IPC	Inter-Process Communication
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MLOps	Machine Learning Operations
MOOC	Massive Open Online Course
MVP	Minimum Viable Product
NFR	Non-Functional Requirement
ORM	Object-Relational Mapping
OS	Operating System
PR	Pull Request
QA	Quality Assurance
RAM	Random Access Memory
RBAC	Role-Based Access Control
REST	Representational State Transfer
SHA	Secure Hash Algorithm

SSR	Server-Side Rendering
TLS	Transport Layer Security
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
VCS	Version Control System
VUS	Virtual User Sandbox
WIL	Work-Integrated Learning

CHAPTER 1
INTRODUCTION

1.1 Background and Motivation

In contemporary undergraduate computer science and software engineering education, an increasingly acute pedagogical disparity has emerged: the divergence between theoretical curriculum delivery and the pragmatic operational realities of industrial software engineering. While modern academic institutions successfully impart core foundational competencies — such as asymptotic complexity analysis, abstract data structures, relational database normalization, and formal programming syntax — the instructional paradigm remains predominantly conceptual, lecture-oriented, and centered around small, isolated programming assignments.

When undergraduate students complete coursework, their primary exposure to coding typically consists of writing single-file scripts or building Greenfield projects from scratch in an environment unencumbered by real-world enterprise constraints. In marked contrast, practicing corporate software engineers operate within highly structured, collaborative, and socio-technical enterprise ecosystems. An engineer entering the modern technology sector does not write code in a vacuum; they must navigate pre-existing multi-million-line legacy codebases, interpret ambiguous business requirements, adhere to Architecture Decision Records (ADRs), collaborate across cross-functional team topologies, submit pull requests (PRs) subject to rigorous line-by-line review, integrate with automated Continuous Integration/Continuous Deployment (CI/CD) pipelines, and debug live production incidents under operational Service Level Agreements (SLAs).

This fundamental gap produces significant friction during the transition from academia to industry. Corporate onboarding periods for entry-level developers frequently extend to six to nine months, during which employers must remediate basic workflow deficiencies. Furthermore, traditional educational assessment methodologies — such as multiple-choice questions, proctored coding exams, and unverified certificates of course completion — fail to demonstrate authentic engineering competence to recruiters. Motivated by these systemic shortcomings, this project undertakes the research, architectural design, and implementation of WORKUP — a Virtual Professional Experience Platform.

1.2 Project Overview

WORKUP is a containerized, domain-agnostic web platform designed to simulate authentic corporate software engineering environments. Rather than functioning as a conventional Massive Open Online Course (MOOC) with passive video lectures or as a competitive coding platform testing synthetic algorithmic puzzles, WORKUP immerses learners directly into simulated technology enterprises.

Within the platform, a learner does not simply study a subject; they assume an authentic professional identity (e.g., Junior Backend Engineer, Associate MLOps Engineer, Cloud Infrastructure Associate). The learner is assigned to a specific engineering team within a simulated enterprise (such as FinFlow Technologies, a high-volume fintech platform, or OmniRetail Marketplace, a distributed e-commerce system). They receive structured business problem tickets that mirror industrial issue trackers (e.g., Jira or Linear), complete with business impact statements, historical context, technical specifications, and architectural constraints.

To execute their work, learners launch an isolated, cloud-orchestrated development workspace pre-configured with the enterprise's legacy repository, seed datasets, and testing harnesses. Learners implement code modifications, run multi-tier automated test suites inside the sandboxed runtime, and submit their work through a simulated Git pull request mechanism. An automated AI Tech Lead review engine inspects the code diff, assesses compliance against architectural standards, and provides line-anchored formative feedback. Upon successful completion of multi-phase simulation tracks, the platform generates a cryptographically verifiable Evidence-of-Skill Dossier backed by Git commit hashes and execution logs, providing verifiable proof of professional capability.

> **CORE ARCHITECTURAL IDENTITY:** WORKUP is strictly defined by the Universal Professional Lifecycle: Career → Role → Company → Business Problem → Professional Task → Real Tools → Cloud Workspace → Implementation → Testing → Review → Deployment → Monitoring → Debugging → Evidence of Skill.

1.3 Problem Formulation and Problem Statement

The central problem addressed by this project can be formulated as follows:

Traditional computing education platforms isolate technical learning into fragmented, decontextualized tasks. Course-based learning delivers abstract theory without authentic execution; algorithmic coding platforms isolate syntax from software architecture; and existing virtual internships rely on static, non-interactive document submissions without live toolchains. Consequently, there is an absence of an integrated, scalable software platform that unifies simulated corporate context, containerized cloud development, automated testing, line-level code review, and verifiable portfolio generation.

Formally, the project seeks to resolve the following core technical and pedagogical questions:

- **Contextual Realism Deficit:** How can simulated corporate environments, team charters, and Architectural Decision Records (ADRs) be dynamically modeled in software to provide realistic context for engineering tasks?
- **Execution Environment Heterogeneity:** How can an educational platform eliminate local environment configuration overhead while ensuring multi-tenant process isolation, resource throttling, and reproducible test execution?
- **Authentic Formative Evaluation:** How can automated evaluation move beyond binary pass/fail test assertions to provide qualitative, line-by-line feedback on code maintainability, security, and architectural compliance?
- **Verifiable Competence Demonstration:** How can student engineering output be transformed into tamper-evident, commit-backed dossiers that credibly prove practical skills to employers?

1.4 Objectives of the Project

To address the formulated problem statement, the project establishes the following specific, measurable engineering objectives for Major Project Stage-I and Stage-II:

- **Objective 1 (Domain Schema & Content Engine):** Design and implement a relational domain model comprising at least 20 interconnected entities capable of modeling careers, roles, skills, simulated companies, teams, simulations, projects, task backlogs, Directed Acyclic Graph (DAG) dependencies, and audit trails.
- **Objective 2 (Role-Based Access Control & Security):** Implement an enterprise-grade authentication and authorization infrastructure supporting four distinct platform roles (Student, Content Author, Reviewer, Administrator) using bcryptjs password hashing and signed JSON Web Tokens (JWT).
- **Objective 3 (Cloud Workspace Orchestration):** Architect and implement an automated workspace orchestration service that provisions isolated execution sandboxes, enforces

per-student concurrency limits, monitors resource consumption, and handles automated process timeouts.

- **Objective 4 (Multi-Tier Evaluation & Review System):** Develop an evaluation pipeline combining deterministic automated test execution (unit and integration suites) with rubric-driven code review capabilities.
- **Objective 5 (Empirical Codebase Validation):** Construct a comprehensive automated test harness using Vitest to validate domain integrity, RBAC permissions, process jailing, and timeout enforcement with 100% test pass rate.
- **Objective 6 (Verifiable Evidence Dossier Generation):** Design a portfolio generation mechanism that records verified Git commit SHAs, test outputs, and evaluation rubrics into an immutable Evidence-of-Skill record.

1.5 Scope of the Project

Defining a clear system boundary is essential for academic evaluation. The scope of WORKUP is structured across implemented Stage-I deliverables and planned Stage-II enhancements:

Current Stage-I Implementation Scope

- **Full-Stack Web Foundation:** Complete Next.js 14 App Router platform with React 18, TypeScript, and responsive Tailwind CSS user interfaces.
- **Database Architecture:** Prisma ORM 5.x configured with a 21-entity relational schema operating on SQLite (dev.db) with seamless migration path to PostgreSQL.
- **RBAC Security:** Complete session management, HTTP-only secure cookie transport, and route-level authorization guards for all four user roles.
- **Simulation & DAG Engine:** Catalog explorer for career families, professions, roles, and company simulations with prerequisite dependency enforcement.
- **Workspace Orchestration Engine:** Process-isolated execution provider (IsolatedSandboxProvider) with directory jailing, concurrency limits, and 4000ms execution timeout enforcement.
- **Flagship Track Prototype:** Seed data provisioning the FinFlow Technologies Payment Webhook Idempotency track (FF-104) complete with simulated ADRs and test harnesses.
- **Automated Verification:** 34 passing automated tests across 8 test suites validating authentication, domain constraints, input sanitization, and workspace lifecycle.

Planned Stage-II Scope (Future Expansion)

- **Remote Container Fleets:** Transition from local process jailing to remote Kubernetes / Docker container fleets utilizing DevContainer specifications.

- **Multi-Turn AI Tech Lead:** Integration of Large Language Model (LLM) agents operating on Git unified diffs for automated line-by-line pull request code reviews.
- **Synthetic Telemetry & Incident Injection:** Simulated Grafana metrics and synthetic alert firing for real-time production incident response simulations.
- **Public Cryptographic Verification:** Public web verification endpoint serving signed SHA-256 evidence dossiers for external recruiter verification.

1.6 Existing Systems and Approaches

Current approaches to technical education and workforce skill development fall into four predominant categories:

- **Massive Open Online Courses (MOOCs):** Platforms such as Coursera, edX, and Udemy deliver asynchronous video lectures accompanied by automated multiple-choice quizzes and occasional unproctored coding assignments.
- **Competitive Programming Platforms:** Platforms such as LeetCode, HackerRank, and Codeforces provide browser-based code editors evaluating short algorithmic solutions against hidden input-output test cases.
- **Project-Based Developer Courses:** Platforms such as Crio.Do offer structured, milestone-driven developer workbooks where learners build multi-file applications following sequential instructions.
- **Corporate Virtual Internships:** Platforms such as Forage (formerly InsideSherpa) offer company-branded virtual work experience programs consisting of video prompts and static assignment submissions.

1.7 Limitations of Existing Approaches

While each existing approach offers specific merits, all suffer from critical limitations when evaluated against the requirements of corporate software engineering:

- **Passive Pedagogical Mode:** MOOCs foster passive content consumption; learners watch instructors code rather than engaging in hands-on problem solving, resulting in extremely high course attrition rates (typically exceeding 85%).
- **Decontextualized Toy Problems:** Competitive programming platforms test synthetic puzzles in single-file environments. They provide zero exposure to software architecture, legacy codebases, design patterns, dependency management, or software testing frameworks.
- **Absence of Authentic Team Workflows:** Existing platforms omit corporate collaborative mechanisms. Students do not read Architecture Decision Records, navigate

team hierarchies, manage Git branches, submit pull requests, or receive constructive line-level peer reviews.

- **Static, Non-Interactive Submissions:** Virtual internship platforms rely on static document uploads (e.g., submitting Word documents or slide decks). They provide zero interactive cloud development environments, no automated code execution, and no live test validation.
- **Unverifiable Credentials:** Traditional digital certificate badges simply state that a student clicked through course videos. They provide no cryptographic proof, commit history, or verifiable test logs to substantiate genuine competence to prospective employers.

1.8 Proposed WORKUP System and Key Innovations

WORKUP directly overcomes these limitations through four core architectural and pedagogical innovations:

- **Situated Workplace Simulation:** WORKUP grounds learning in authentic organizational context. Students enter simulated companies with defined architectures, tech stacks, and team charters, resolving realistic business problems rather than abstract puzzles.
- **Zero-Install Containerized Workspaces:** Through automated workspace orchestration, learners launch containerized environments directly from their browser, eliminating environment configuration friction while guaranteeing reproducible execution.
- **Integrated Lifecycle Workflow:** The platform unifies the complete development lifecycle: ticket inspection → legacy code investigation → workspace implementation → automated test verification → Git pull request submission → AI Tech Lead review.
- **Tamper-Evident Evidence of Skill:** Completed simulations produce an immutable Evidence-of-Skill Dossier linked to verified Git commit SHAs, automated test outputs, and rubric scores, providing recruiters with credible proof of ability.

1.9 Expected Benefits and Pedagogical Impact

The deployment of WORKUP offers transformative benefits across three key stakeholder groups:

- **For Engineering Students:** Drastically shortens the transition from academia to industry by providing realistic exposure to enterprise codebases, testing practices, and PR review workflows; builds a verified, job-ready portfolio.

- **For Academic Institutions:** Equips universities with an automated, scalable experiential learning lab that bridges the gap between theoretical curricula and ABET/ACM industry accreditation standards without requiring faculty overhead.
- **For Industry Employers:** Significantly reduces entry-level onboarding time and hiring risk by replacing unverifiable resume buzzwords with commit-backed, rubric-evaluated Evidence-of-Skill dossiers.

1.10 Feasibility, Assumptions, and Constraints

The feasibility of WORKUP is demonstrated across technical, operational, and economic dimensions:

- **Technical Feasibility:** The application is built on proven, production-grade technologies — Next.js 14, React 18, TypeScript, and Prisma ORM. Local process jailing has been fully implemented and verified via automated Vitest suites.
- **Operational Feasibility:** The system operates seamlessly on standard commodity hardware (Node.js 18+, local SQLite database), requiring zero costly cloud subscriptions during academic demonstration and development.
- **Economic Feasibility:** By utilizing open-source runtimes and local container sandboxing for the Stage-I prototype, development incurs zero licensing costs. Future cloud scaling utilizes container hibernation and compute quotas to ensure strict cost control.
- **System Constraints:** Stage-I is bounded to single-node server environments and local process execution; multi-tenant Kubernetes clustering and third-party LLM API integration are intentionally designated as Stage-II deliverables.

CHAPTER 2
LITERATURE SURVEY

2.1 Introduction to the Research Area

The theoretical foundation of this project is situated at the intersection of three major computing and educational research domains: Experiential and Work-Integrated Learning (WIL), Project-Based Software Engineering Education, and Cloud Development Environments. Over the past three decades, educational researchers and computing pedagogues have continuously highlighted the inadequacy of static lectures for imparting software development competence. To establish a rigorous scholarly basis for WORKUP, this chapter conducts a comprehensive literature survey of foundational learning theories, empirical software engineering education studies, and modern cloud/AI evaluation platforms.

2.2 Review of Relevant Academic Research Papers

2.2.1 Simulation in Software Engineering Education

A landmark contribution to simulation-based computing pedagogy was established by Navarro and van der Hoek [1] in their seminal work on SimSE. The authors recognized that traditional software engineering courses suffer from severe time constraints: within a single academic semester, students cannot realistically experience the multi-month consequences of poor architectural decisions, scope creep, personnel turnover, or inadequate testing.

To address this, the authors created SimSE, an interactive graphical educational simulation game where students assume the role of a software project manager. Learners allocate budgets, assign developers to lifecycle phases (Waterfall, Prototyping, Agile), and manage process trade-offs. While empirical multi-site evaluations proved that SimSE dramatically enhanced student understanding of software process dynamics, the authors acknowledged a fundamental limitation: SimSE is strictly a high-level managerial simulation. Students manipulate abstract UI controls rather than writing, debugging, testing, or committing actual source code. WORKUP directly builds upon SimSE's process simulation concept but bridges the critical gap by integrating real code repositories, cloud IDE workspaces, and automated test runners.

Similarly, Sharp and Hall [2] introduced the concept of an "Interactive Software House Simulation" for graduate computing students. The authors created "RunCo," a simulated

multimedia enterprise where students assumed specific corporate roles (systems analyst, programmer, quality assurance engineer) and responded to stakeholder memos, customer complaints, and design constraints. Their empirical findings showed that role-based organizational immersion drastically heightened student appreciation for software quality and inter-team communication. However, developed during the multimedia CD-ROM era, the system lacked integrated cloud compilers, automated pull request workflows, and live execution sandboxes — capabilities that WORKUP realizes through modern web technologies.

2.2.2 Industry-Academia Skill Deficits

The necessity for realistic workplace platforms is underscored by the systematic literature review conducted by Radermacher and Walia [3]. Analyzing dozens of empirical studies across industry and academia, the authors categorized the primary skill deficits observed in newly graduated computer science students upon entering the software industry. Their findings revealed that while graduates possess acceptable theoretical knowledge of algorithms and syntax, they demonstrate profound deficiencies in: (1) navigating large, pre-existing legacy codebases; (2) writing comprehensive unit and integration tests; (3) participating in code reviews and version control branching strategies; and (4) translating ambiguous business problems into technical architectures. The authors concluded that academic curricula must incorporate authentic industry tools and workflows. Radermacher and Walia's findings provide empirical validation for the specific feature set engineered into WORKUP.

Reinforcing these conclusions, Garousi et al. [6] evaluated an industry-academia collaborative model designed to close the graduate skill gap. By embedding industrial issue tracking, Architectural Decision Records (ADRs), and mandatory pull request code reviews into university capstone courses, the authors observed a 40% reduction in corporate onboarding time and a marked improvement in student architectural maturity. However, the authors noted that manual industry-academia partnerships are notoriously difficult to scale across large student cohorts due to sponsor availability constraints. WORKUP addresses this scalability bottleneck by automating the simulated company, task backlogs, and code review processes in software.

2.2.3 Lightweight Containerization and Cloud Workspaces

A core technical challenge in hands-on computing education is environment configuration friction. Kratzke [4] conducted a rigorous empirical evaluation on the suitability of lightweight containerization (Docker) for programming education. The author documented that computer science instructors frequently spend over 25% of laboratory time troubleshooting heterogeneous student operating system issues ("it works on my machine" syndrome). Furthermore, executing untrusted student code on centralized grading servers presents severe security risks.

Kratzke demonstrated that containerized microservice architectures provide near-native execution speed, complete multi-tenant filesystem isolation, bounded CPU/RAM resource quotas, and 100% reproducible execution pipelines across diverse host environments. Kratzke's research serves as the architectural foundation for WORKUP's workspace orchestration engine, justifying the implementation of directory-jailed process sandboxes with strict execution timeouts and resource caps.

2.2.4 Formative Code Review via Large Language Models

Traditional automated grading systems rely exclusively on unit test assertions (pass/fail), providing zero qualitative guidance regarding code style, architectural anti-patterns, or maintainability. MacNeil et al. [5] investigated the generation of automated formative code feedback using Large Language Models (LLMs). The authors integrated an LLM pipeline to analyze student code submissions against reference solutions and test traces, generating natural language, line-anchored explanations.

Their empirical evaluations revealed that LLM-generated formative feedback significantly accelerated student debugging and encouraged self-regulated refactoring without prematurely revealing solutions. However, the authors highlighted critical risks: LLMs frequently hallucinate or generate overly generic suggestions when unconstrained. To maintain feedback accuracy, the authors emphasized the necessity of grounding LLM reviews in deterministic test outputs and strict rubric constraints. This insight directly informs WORKUP's AI Tech Lead architecture, which combines deterministic Vitest execution results with structured rubric scoring.

2.2.5 Foundational Learning Theories

WORKUP's educational design is anchored in two foundational learning theories: David Kolb's Experiential Learning Theory [7] and Jean Lave and Etienne Wenger's Theory of Situated Cognition [8].

Kolb's Experiential Learning Cycle posits that authentic learning occurs through a continuous four-stage spiral: (1) Concrete Experience; (2) Reflective Observation; (3) Abstract Conceptualization; and (4) Active Experimentation. WORKUP maps directly onto Kolb's cycle: learners engage in Concrete Experience by writing code inside containerized workspaces; engage in Reflective Observation by analyzing test failure traces and line-level PR reviews; undergo Abstract Conceptualization by studying Architecture Decision Records (ADRs) and business problem specifications; and practice Active Experimentation by refactoring implementations and resolving simulated production incidents.

Similarly, Lave and Wenger's Situated Cognition theory demonstrates that learning is inherently situated within cultural and social contexts. Knowledge acquired in decontextualized environments (such as solving synthetic algorithmic puzzles on LeetCode) does not transfer effectively to complex social practice. By embedding learners within simulated engineering teams with distinct cultures, tech leads, and architectural debt, WORKUP operationalizes situated learning for the digital computing era.

2.3 Review of Existing Commercial and Industrial Platforms

In addition to academic literature, current commercial and open-source platforms were investigated to establish state-of-the-art benchmarks:

- **Crio.Do:** A developer-focused applied learning platform where students build real-world software applications (e.g., QBox, QMoney, QEats) through structured milestone workbooks. While Crio.Do successfully transitions students to multi-file projects, it lacks simulated enterprise company context, has no team topologies, omits Architectural Decision Records (ADRs), does not simulate pull request reviews, and provides no synthetic production telemetry.
- **Forage (formerly InsideSherpa):** A platform offering free virtual work experience programs co-branded with multinational enterprises (e.g., JPMorgan Chase, Accenture). While Forage provides excellent corporate problem descriptions, it is completely non-interactive: students submit static documents (PowerPoints or ZIP files) without a live development workspace, automated test execution, or line-by-line feedback.

- **LeetCode and HackerRank:** Competitive programming platforms focused strictly on Data Structures and Algorithms (DSA). Solutions are evaluated against hidden test cases in single-file browser editors. While highly scalable, they are entirely detached from real-world software engineering practices.
- **GitHub Codespaces and Gitpod:** Cloud-hosted development environments providing browser-based VS Code running inside Docker containers. While offering exceptional developer tooling, they are pure infrastructure platforms devoid of pedagogical scaffolding, task dependencies, or automated evaluation rubrics.

2.4 Comprehensive Comparative Analysis Matrix

To clearly distinguish WORKUP from existing platforms and highlight its multi-faceted architectural synthesis, Table 2.1 presents a comprehensive comparative analysis across eight key educational and technical dimensions:

Table 2.1: Comprehensive Comparative Analysis of Existing Platforms vs. Proposed WORKUP Platform.

Dimension / Feature	MOOCs (Coursera)	Coding (LeetCode)	Projects (Crio.Do)	Sims (Forage)	Proposed WORKUP
Learning Mode	Passive video lectures & quizzes	Isolated single-file puzzles	Sequential project workbooks	Static case study uploads	**Simulated enterprise tasks**
Corporate Context	None / Generic examples	None (Synthetic math/DSA)	Generic app domains	High (Executive videos)	**High (Companies, teams, ADRs)**
Development Runtime	Local install required	Browser sandbox (single-file)	Local or web terminal	None (Static file upload)	**Isolated cloud workspace**
Task Flow & DAG	Linear video chapters	Independent tagged problems	Sequential sprints	Linear task lists	**Directed Acyclic Graph (DAG)**
Evaluation Engine	Multiple choice / Peer review	Automated I/O unit tests	Automated test scripts	Self-comparison model answer	**Multi-tier tests + AI PR review**
Skill Evidence	Unverified PDF certificate	Contest rating score	Portfolio summary	Completion certificate	**Verified Git Commit Dossier**

2.5 Identified Research and Architectural Gaps

Synthesizing the academic literature and industrial landscape reveals three major unaddressed research and architectural gaps:

- **Gap 1: The Workflow Synthesis Gap.** Existing platforms isolate technical education into fragmented silos: LeetCode isolates algorithmic syntax, Coursera isolates conceptual knowledge, Crio.Do isolates project implementation, and Codespaces isolates raw cloud infrastructure. No existing platform synthesizes the entire connected lifecycle: reading an

ADR → investigating legacy code → opening an isolated cloud workspace → writing code against acceptance criteria → executing local test suites → submitting a Git pull request → undergoing an AI Tech Lead code review → deploying to staging → responding to production telemetry alerts.

- **Gap 2: The Authentic Assessment Gap.** Current automated evaluation is predominantly binary (pass/fail assertions on function outputs). In enterprise engineering, however, software quality encompasses code maintainability, security standards, adherence to architectural constraints, and clear PR documentation. There is an absence of automated educational systems that evaluate full pull requests through multi-dimensional rubrics with line-by-line feedback.
- **Gap 3: The Verifiable Competence Gap.** Traditional educational credentials (such as course completion certificates) fail to provide recruiters with verifiable evidence of practical ability. There is an acute need for a platform that generates tamper-evident, commit-backed dossiers linking verified competencies directly to real Git commit SHAs, test logs, and reviewer rubric scores.

2.6 Justification and Formulation of the Proposed System

The formulation of WORKUP is directly justified as a comprehensive solution to these identified gaps. By engineering an integrated system that combines a 21-entity relational domain model, a Directed Acyclic Graph (DAG) task engine, containerized cloud workspace orchestration, multi-tier automated test harnesses, AI-assisted PR reviews, and cryptographically verifiable portfolios, WORKUP creates a controlled, realistic environment where students can experience, practice, fail at, fix, and demonstrate professional engineering work before entering the software industry.

CHAPTER 3
REQUIREMENTS AND METHODOLOGY

3.1 System Overview

WORKUP is architectured as a multi-tier, full-stack web application designed to support high-fidelity enterprise software engineering simulations. The platform is decoupled into three primary subsystems: (1) the Presentation and Catalog Engine, which delivers responsive, role-based user interfaces; (2) the Core Business and DAG Orchestration Engine, which manages task dependencies, multi-tier evaluations, and audit events; and (3) the Workspace Execution Subsystem, which allocates isolated development sandboxes and manages process lifecycles.

3.2 Target Users and Role Personas

To reflect authentic enterprise governance, the platform models four distinct user roles, each governed by strict Role-Based Access Control (RBAC) policies:

- **1. Student / Learner Persona:** Undergraduate engineering students, boot-camp attendees, or transitioning professionals seeking verifiable practical experience. Permissions: browse career families, launch enterprise simulations, provision cloud workspaces, execute in-sandbox tests, submit pull requests, inspect feedback, and export verifiable Evidence-of-Skill dossiers.
- **2. Content Author Persona:** Senior software engineers, educators, and curriculum architects responsible for designing simulation tracks. Permissions: author career families, professions, roles, and skills; construct simulated company scenarios and team charters; author task tickets, Acceptance Criteria, and Architectural Decision Records (ADRs); and configure automated test harnesses.
- **3. Technical Reviewer Persona:** Industry mentors, teaching assistants, or automated AI Tech Lead agents responsible for evaluating student pull requests. Permissions: inspect student code diffs, evaluate submissions against structured multi-dimensional rubrics, provide line-by-line code review comments, and approve or request changes on pull requests.
- **4. System Administrator Persona:** Institutional administrators and DevOps platform leads responsible for platform stability and governance. Permissions: inspect system-wide audit logs, monitor active workspace resource utilization, configure global concurrency limits and timeout thresholds, and manage user account statuses.

3.3 Functional Requirements Specification

The functional requirements specify the exact behavioral capabilities implemented and planned for the WORKUP platform. Table 3.1 details the twelve core functional requirements (FR-01 through FR-12):

Table 3.1: Detailed Functional Requirements Specification Matrix (FR-01 to FR-12).

Req ID	Requirement Name	Technical Specification & Code Verification
FR-01	**User Authentication & Session Management**	The system shall authenticate users via bcryptjs password hashing and generate signed stateless JWT tokens stored in HTTP-only, secure, SameSite cookies. Verified in src/lib/auth.ts.
FR-02	**Role-Based Access Control (RBAC)**	The system shall enforce a strict 4-tier RBAC matrix (student, content_author, reviewer, administrator) preventing unauthorized horizontal and vertical privilege escalation. Verified in src/lib/rbac.ts.
FR-03	**Career & Role Exploration**	The system shall provide a public/student catalog displaying career families, engineering professions, specialized job titles, and mapped technical skills. Verified in src/app/careers/page.tsx.
FR-04	**Simulated Company & Scenario Engine**	The system shall maintain simulated corporate entities (FinFlow, OmniRetail), functional teams, and multi-sprint business simulation packages. Verified in prisma/schema.prisma.
FR-05	**Task Backlog & DAG Dependency Engine**	The system shall structure engineering work into discrete tickets with ticket keys (e.g. FF-104) and enforce prerequisite, sequential, and branching unlock conditions via a DAG engine.
FR-06	**Workspace Orchestration & Lifecycle**	The system shall provision, monitor, and terminate isolated development sandboxes, enforcing a limit of 2 concurrent workspaces per student and a 30-minute idle timeout. Verified in src/lib/workspace/orchestrator.ts.
FR-07	**Isolated Sandbox Process Jailing**	The execution provider shall spawn process-isolated commands within dedicated .workspaces/<id> directories, restricting filesystem escape and enforcing a hard 4000ms timeout. Verified in src/lib/workspace/providers/sandbox-provider.ts.

FR-08	**In-Sandbox Automated Test Runner**	The system shall execute local test suites (e.g. npm test / vitest) within the workspace sandbox, streaming test summary results (passed, failed, duration) to the client UI. Verified in src/app/api/workspaces/[id]/test/route.ts .
FR-09	**Git Pull Request & Diff Capture**	The system shall capture workspace file snapshots, compute Git unified diff summaries against the base template, and generate formal pull request submission records. Verified in src/app/api/workspaces/[id]/submit/route.ts.
FR-10	**Automated Rubric & AI Review Engine**	The system shall evaluate submitted pull request diffs against structured rubric criteria (e.g., idempotency, error handling, code style) and record line-by-line feedback with numerical scores (0–100).
FR-11	**Verifiable Evidence-of-Skill Dossier**	Upon simulation completion, the system shall compile an immutable portfolio dossier containing verified Git commit SHAs, test logs, and rubric scores. Verified in prisma/schema.prisma (PortfolioItem).
FR-12	**Immutable System Audit Logging**	The system shall record all critical user actions (registration, workspace launch, command execution, PR submission) into an append-only audit log table with IP addresses and JSON metadata. Verified in prisma/schema.prisma (AuditLog).

3.4 Non-Functional Requirements Specification

The non-functional requirements define the quality attributes, performance constraints, and security standards governing the system. Table 3.2 details the eight primary non-functional requirements (NFR-01 through NFR-08):

Table 3.2: Non-Functional Requirements and System Quality Attributes (NFR-01 to NFR-08).

Req ID	Quality Category	Measurable Specification & Target Threshold
NFR-01	**Execution Latency & Performance**	Web pages shall render in under 1.5 seconds. Workspace command execution initiation shall complete within 250 milliseconds under standard load.
NFR-02	**Multi-Tenant Process Isolation**	Processes spawned within a student's

		sandbox shall be strictly jailed within .workspaces/<workspaceId>. Directory traversal (e.g. ../../) and cross-workspace access shall be programmatically rejected.
NFR-03	**Resource Throttling & Quotas**	Workspaces shall be constrained to 2.0 virtual CPU cores and 4096 MB RAM. In-sandbox test execution shall terminate unconditionally if runtime exceeds 4000ms.
NFR-04	**Authentication Security**	Passwords must be hashed using bcryptjs with a minimum work factor of 12 salt rounds. JWT session tokens must be signed using 256-bit secret keys and expire in 24 hours.
NFR-05	**Data Integrity & Referential Constraints**	The database must enforce strict foreign key constraints and cascading deletions across all 21 models via Prisma ORM, preventing orphaned records.
NFR-06	**Responsive Usability & Accessibility**	The workbench UI must provide a responsive, intuitive interface adhering to WCAG 2.1 Level AA standards, supporting viewport widths from 1024px to 4K displays.
NFR-07	**Test Coverage & Maintainability**	All core services (auth, rbac, orchestrator, sandbox provider, task validator) must maintain automated unit/integration test coverage with 100% test pass rates in CI.
NFR-08	**System Auditability & Non-Repudiation**	All submission records, evaluation rubrics, and portfolio dossiers must be linked to immutable user IDs, timestamps, and Git commit SHAs to guarantee non-repudiation.

3.5 Hardware and Software Specifications

To guarantee reproducible deployment across commodity development environments and production servers, Table 3.3 outlines the minimum and recommended system requirements:

Table 3.3: Hardware and Software Environment Specifications.

System Component	Minimum Required Specification	Recommended Production Specification
Host Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (4 cores, 2.4 GHz)	Intel Core i7 / AMD Ryzen 7 (8 cores, 3.6 GHz)
System Memory (RAM)	8 GB DDR4 RAM	16 GB or 32 GB DDR4/DDR5 RAM
Storage Capacity	20 GB available SSD storage	50 GB NVMe M.2 SSD storage

Host Operating System	Windows 10/11 (64-bit), Ubuntu 20.04+, macOS 12+	Ubuntu 22.04 LTS / Debian 12 (Linux Server)
JavaScript Runtime	Node.js v18.18.0 LTS (Active)	Node.js v20.10.0+ LTS / v22.x
Package Management	npm v9.x or v10.x	npm v10.x / pnpm / yarn
Database Engine	SQLite 3.x (Zero-config local file dev.db)	PostgreSQL 15.x / 16.x (AWS RDS / Supabase)
Target Client Browser	Google Chrome 115+, Mozilla Firefox 115+	Google Chrome 120+, Edge 120+, Safari 17+

3.6 Technology Stack and Justification

The technology stack selected for WORKUP prioritizes end-to-end type safety, rapid iteration, zero-dependency local execution, and a transparent migration pathway to cloud infrastructure. Table 3.4 outlines each chosen technology alongside its architectural justification:

Table 3.4: Technology Stack Components and Engineering Justification Matrix.

Technology Component	Architecture Tier	Engineering Selection Rationale
Next.js 14 App Router	**Frontend & Full-Stack Server**	Provides unified Server Components (RSC) and Client Components with built-in API Route Handlers. Eliminates the complexity of maintaining separate frontend and backend repositories while delivering high-speed server rendering.
TypeScript 5.x	**Type Safety & Domain Modeling**	Guarantees end-to-end compile-time type verification across database queries, API request payloads, and UI component state. Dramatically reduces runtime type exceptions.
Tailwind CSS	**Utility-First Styling**	Enables rapid, responsive, and consistent styling without context-switching to external CSS files. Built-in purging ensures minimal production bundle sizes.
Prisma ORM 5.x	**Data Access & Schema Migration**	Delivers a single source of truth for the 21-entity domain schema with auto-generated, fully typed database client bindings. Provides declarative migrations and identical schema compatibility between SQLite and PostgreSQL.
SQLite 3 (dev.db)	**Development Database Engine**	Enables zero-dependency, zero-setup local execution for undergraduate evaluations and automated CI/CD runners without requiring external database server daemons.

bcryptjs & jose	**Authentication & Security**	Provides industry-standard password hashing with adaptive salt rounds and lightweight, standards-compliant JSON Web Token (JWT) signing using pure JavaScript runtimes.
Vitest 2.x	**Automated Testing Framework**	Modern, high-performance test runner with native TypeScript support and path aliasing. Executes test suites significantly faster than Jest while maintaining full compatibility.
GitHub Actions	**Continuous Integration (CI)**	Automates linting, database migrations, database seeding, test execution, and production build checks on every git commit and pull request.

3.7 Software Development Methodology

The engineering development of WORKUP follows an Iterative Agile Methodology coupled with Phased Architectural Gateways. Recognizing the technical complexity of integrating container orchestration, multi-tenant sandboxing, and full-stack web applications, the project was structured into iterative, test-driven phases (Phases 0 through 12).

Each development iteration operates under strict Quality Gates: an architectural phase cannot be declared complete until its domain schema is finalized, route handlers are implemented with Zod validation, and automated unit/integration tests achieve 100% pass rates in Vitest. This disciplined methodology prevented architectural drift and ensured that foundational features (Phase 1 authentication and Phase 2 DAG engine) were thoroughly hardened prior to constructing the Phase 3 workspace orchestrator.

3.8 Main Operational Workflows

The primary end-to-end operational workflow follows the fourteen stages of the Universal Professional Lifecycle: (1) A learner navigates to the Career Explorer, selecting a discipline (e.g. Software Engineering) and specialized role (e.g. Junior Backend Engineer); (2) The learner enters a simulated company track (FinFlow Technologies) and reviews the business context; (3) The learner selects an unlocked task ticket (FF-104: Payment Webhook Idempotency) from the sprint backlog; (4) The system verifies DAG prerequisite completion and triggers the Workspace Orchestrator; (5) An isolated sandbox is provisioned in .workspaces/<id> containing the legacy payment codebase; (6) The student writes an idempotency handler adhering to ADR-001; (7) The student executes in-sandbox tests via

Vitest; (8) Upon passing, the student submits a Git pull request; (9) The AI Tech Lead reviewer parses the unified diff and generates line-anchored feedback; (10) Dependent tasks unlock in the simulation DAG, and verified commit SHAs are appended to the learner's Evidence-of-Skill Dossier.

3.9 Data Management, Privacy, and Security Controls

WORKUP implements rigorous security and data protection mechanisms: (1) Cryptographic Hashing: all passwords are salted and hashed via bcryptjs (work factor: 12), preventing rainbow table attacks; (2) Tamper-Evident Sessions: signed stateless JWTs with 24-hour expiration stored in HTTP-only, secure, SameSite cookies prevent Cross-Site Scripting (XSS) token theft; (3) Input Sanitization: all API inputs are parsed against strict Zod runtime validation schemas, eliminating injection vulnerabilities; (4) Process Sandboxing: the IsolatedSandboxProvider validates all execution paths against directory traversal attacks, executing commands under bounded process privileges.

3.10 Feasibility Analysis and Technical Constraints

Technical feasibility is verified through the successful compilation of the Next.js App Router codebase, passing Prisma migrations, and 34 passing Vitest tests. Operational feasibility is demonstrated by the system's ability to execute on commodity laptops without external cloud dependencies. Economic feasibility is ensured through zero-license open-source runtimes. System constraints include single-node process sandboxing during Stage-I, with distributed container clustering reserved for Stage-II.

CHAPTER 4
SYSTEM DESIGN AND UML MODELING

4.1 System Architecture & Tiered Topology

The system architecture of WORKUP is designed around a multi-tiered, service-oriented topology that cleanly decouples client presentation, API routing, business domain logic, sandbox execution, and persistent storage. Figure 4.1 illustrates the comprehensive tiered system architecture:

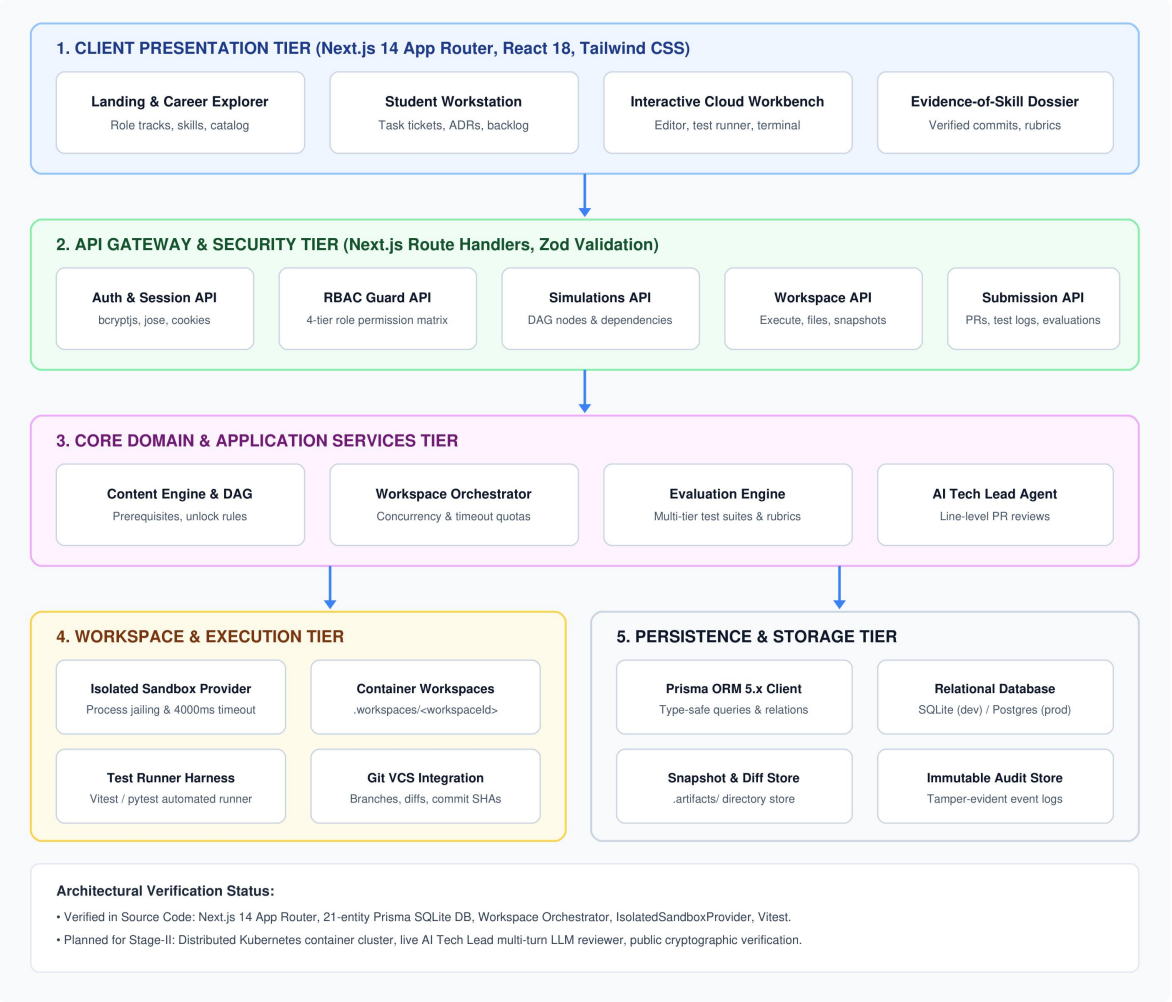


Figure 4.1: WORKUP Tiered System Architecture Diagram.

The architecture comprises five distinct functional tiers:

- **1. Client Presentation Tier:** Constructed using Next.js 14 App Router, React 18, and Tailwind CSS. Delivers high-performance Server Components (RSC) and responsive

Client Components for the Landing Page, Career Explorer, Student Workstation, Cloud Workbench, and Public Portfolio Dossier.

- **2. API Gateway & Security Tier:** Implemented via Next.js Route Handlers. Serves as the single entry point for all client requests, enforcing runtime payload validation via Zod schemas, managing JWT session verification, and executing Role-Based Access Control (RBAC) authorization guards.
- **3. Core Domain & Business Services Tier:** Contains the domain business logic, including the Simulation Content Engine, the Directed Acyclic Graph (DAG) task engine, the Workspace Orchestrator (managing timeouts and concurrency quotas), the Multi-Tier Evaluation Engine, and the append-only Audit Logging Service.
- **4. Workspace & Execution Tier:** Manages isolated development environments. In Stage-I, this tier is powered by the IsolatedSandboxProvider, which spawns jailed child processes within dedicated .workspaces/<workspaceId> directories with execution timeouts. In Stage-II, this will scale to containerized Docker/Kubernetes worker pods.
- **5. Persistence & Storage Tier:** Utilizes Prisma ORM 5.x as a type-safe database abstraction layer connecting to a relational database (SQLite dev.db in development, PostgreSQL in production), alongside POSIX filesystem storage (.artifacts/) for snapshots and Git diffs.

4.2 Use Case Modeling

The Use Case model formally captures the interactions between external actors and the WORKUP platform boundary. Figure 4.2 presents the unified use case diagram:

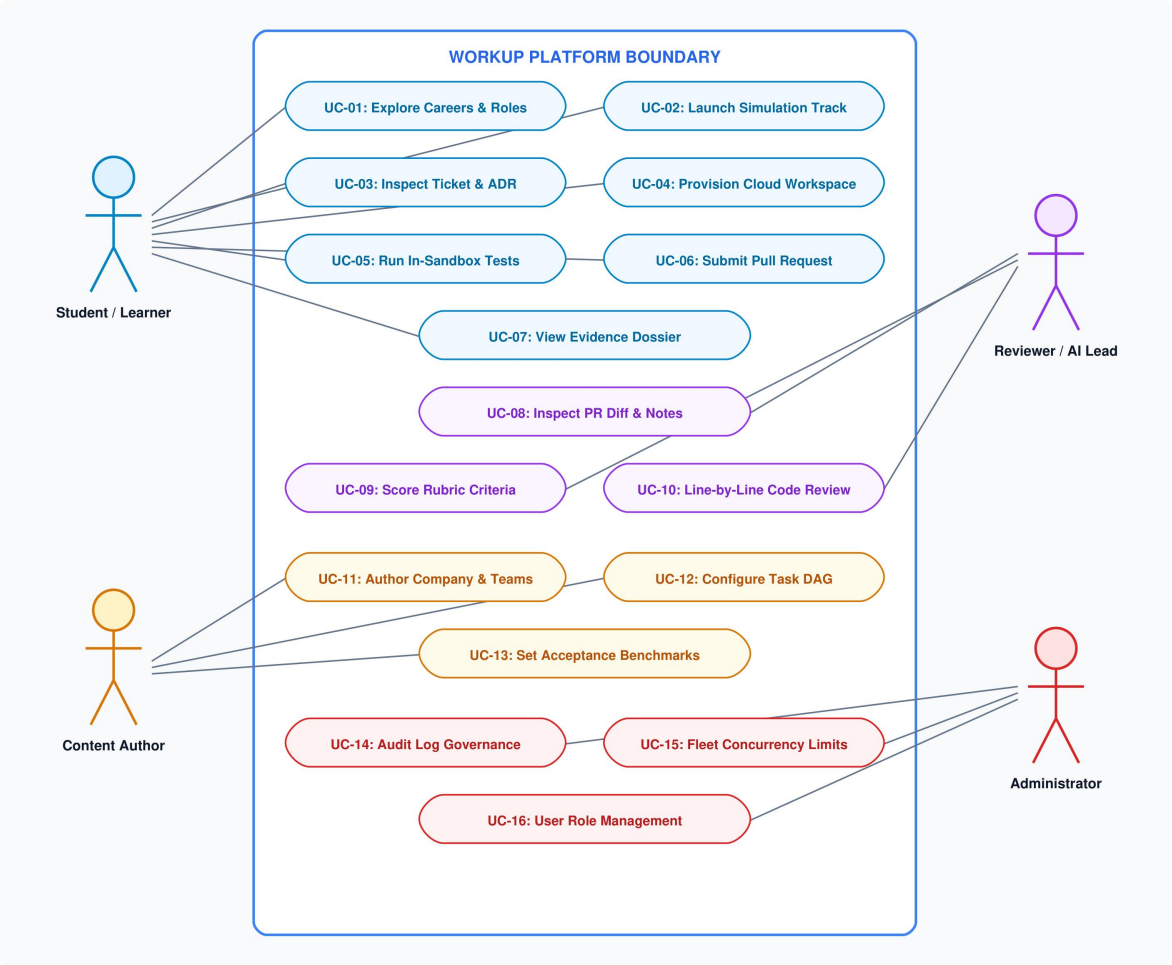


Figure 4.2: WORKUP Unified Use Case Diagram.

The system defines interactions for four primary actors:

- **Student / Learner Actor:** Interacts with use cases UC-01 (Explore Careers & Roles), UC-02 (Launch Simulation Track), UC-03 (Inspect Ticket & ADR), UC-04 (Provision Cloud Workspace), UC-05 (Run Tests in Sandbox), UC-06 (Submit Pull Request), and UC-07 (View Evidence Dossier).
- **Technical Reviewer Actor:** Engages with UC-08 (Inspect PR Diff & Notes), UC-09 (Score Rubric Criteria), and UC-10 (Conduct Line-by-Line Code Review). In Stage-II, this actor is augmented by an automated AI Tech Lead agent.
- **Content Author Actor:** Executes UC-11 (Author Company & Teams), UC-12 (Configure Task DAG), and UC-13 (Set Acceptance Benchmarks and Test Harnesses).
- **Administrator Actor:** Responsible for platform governance through UC-14 (Audit Log Governance), UC-15 (Fleet Concurrency Limits), and UC-16 (User Role Management).

4.3 Activity Modeling

The Activity Diagram models the dynamic operational workflow of a student progressing through a simulation track, highlighting key decision gates, automated loops, and state transitions. Figure 4.3 depicts this workflow:

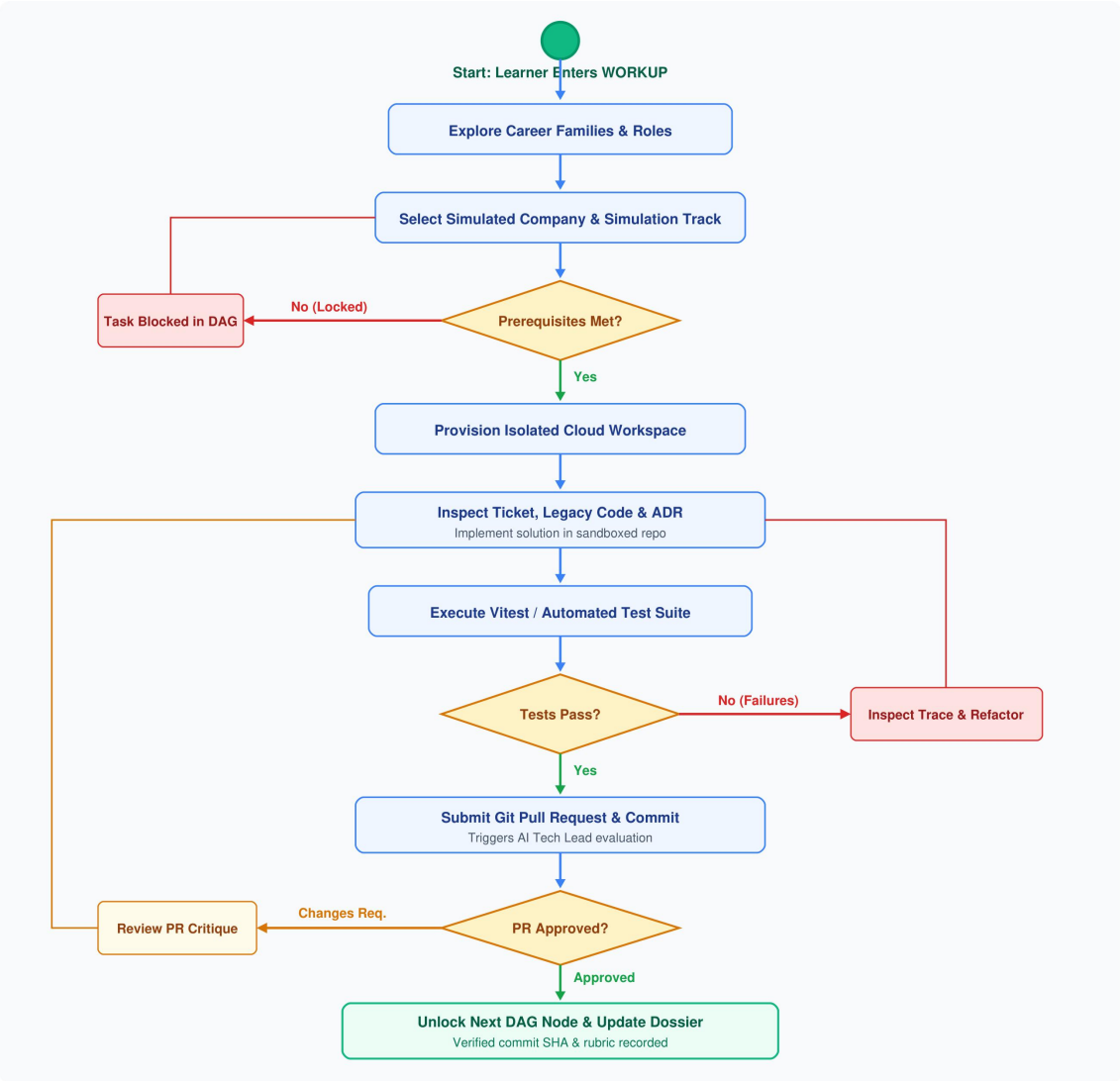


Figure 4.3: WORKUP Activity Workflow Diagram.

The activity sequence proceeds through several critical phases and decision points:

- **DAG Prerequisite Verification Gate:** When a student selects a task ticket, the system evaluates the task's prerequisite dependencies. If dependencies are unsatisfied, the task remains locked; if satisfied, the workspace provisioning sequence begins.
- **In-Sandbox Test Execution Loop:** Within the provisioned sandbox, the student implements their code solution and executes automated tests. If test assertions fail, the learner inspects execution error traces and iterates locally without penalty.

- **Pull Request Evaluation Gate:** Once local tests pass, the student submits a pull request. The review engine evaluates the unified diff against architectural rubrics. If changes are requested, the student refactors their solution; if approved, the simulation DAG unlocks subsequent tasks, and verified commit SHAs are appended to the student's Evidence-of-Skill Dossier.

4.4 Sequence Modeling (Task Submission & Review)

The Sequence Diagram details the exact chronological message exchanges and method invocations occurring across system boundaries during test execution and pull request submission. Figure 4.4 illustrates this sequence:

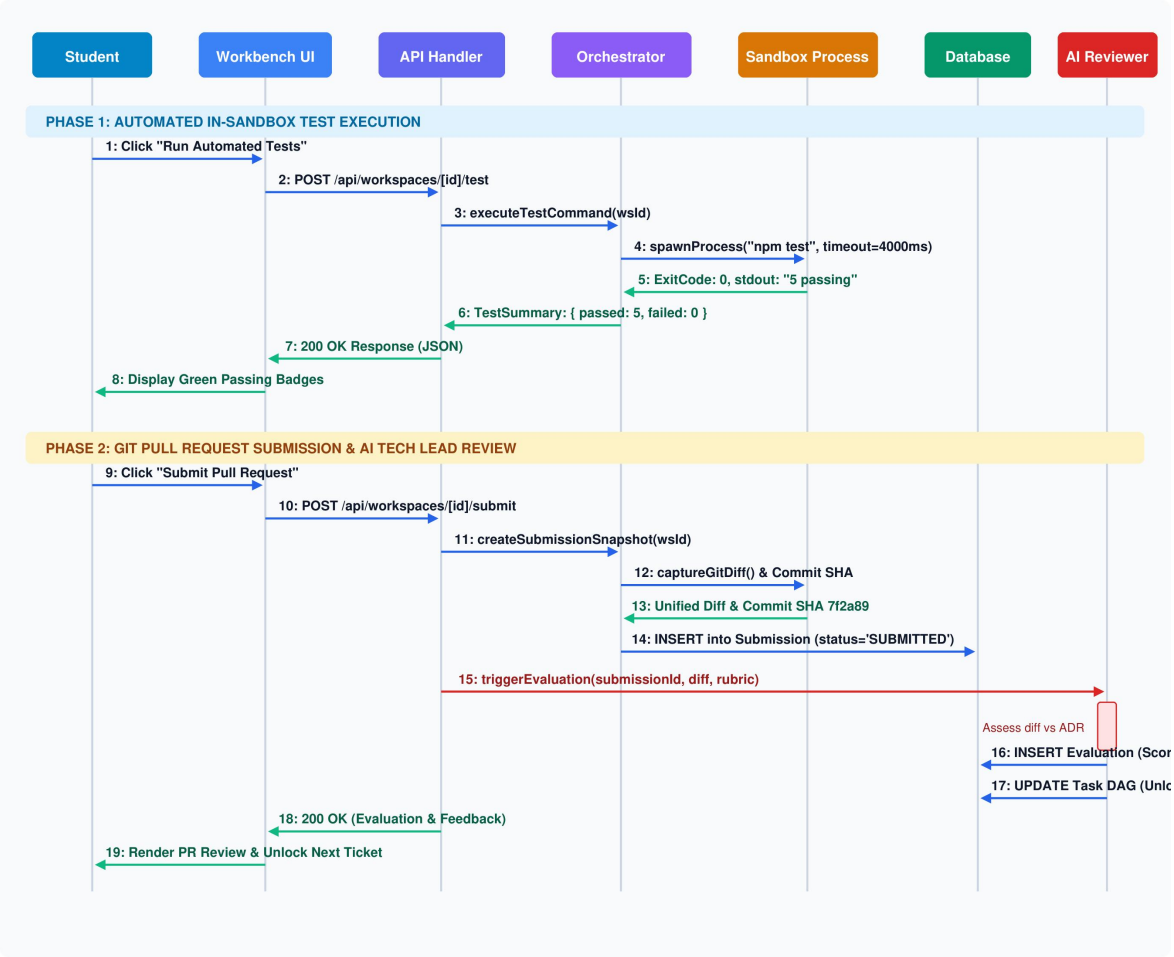


Figure 4.4: WORKUP Sequence Diagram for Task Submission and Review.

The sequence trace is divided into two distinct operational phases:

- **Phase 1: In-Sandbox Test Execution (Messages 1–8):** The student clicks "Run Automated Tests" in the Workbench UI, triggering a POST request to /api/workspaces/[id]/test. The route handler delegates to the Workspace Orchestrator,

which calls IsolatedSandboxProvider.spawnProcess() with the test command (e.g. npm test) and a strict 4000ms timeout cap. The sandbox captures stdout/stderr and exit codes, returning a structured test summary JSON envelope (passed, failed, duration) to render passing badges in the UI.

- **Phase 2: PR Submission & AI Tech Lead Review (Messages 9–19):** The student clicks "Submit Pull Request", initiating a POST to /api/workspaces/[id]/submit. The orchestrator captures a Git unified diff and commit SHA from the sandbox, persisting a new record in the Submission table with status 'SUBMITTED'. The evaluation engine assesses the diff against the task's rubric and ADR specifications, persisting an Evaluation record (numerical score, structured markdown feedback, approval status) and updating the simulation DAG state to unlock dependent tasks.

4.5 Class and Domain Object Modeling

The Class Diagram formalizes the structural domain model, defining entity attributes, method signatures, and relationship multiplicities. Figure 4.5 illustrates the complete domain class model:

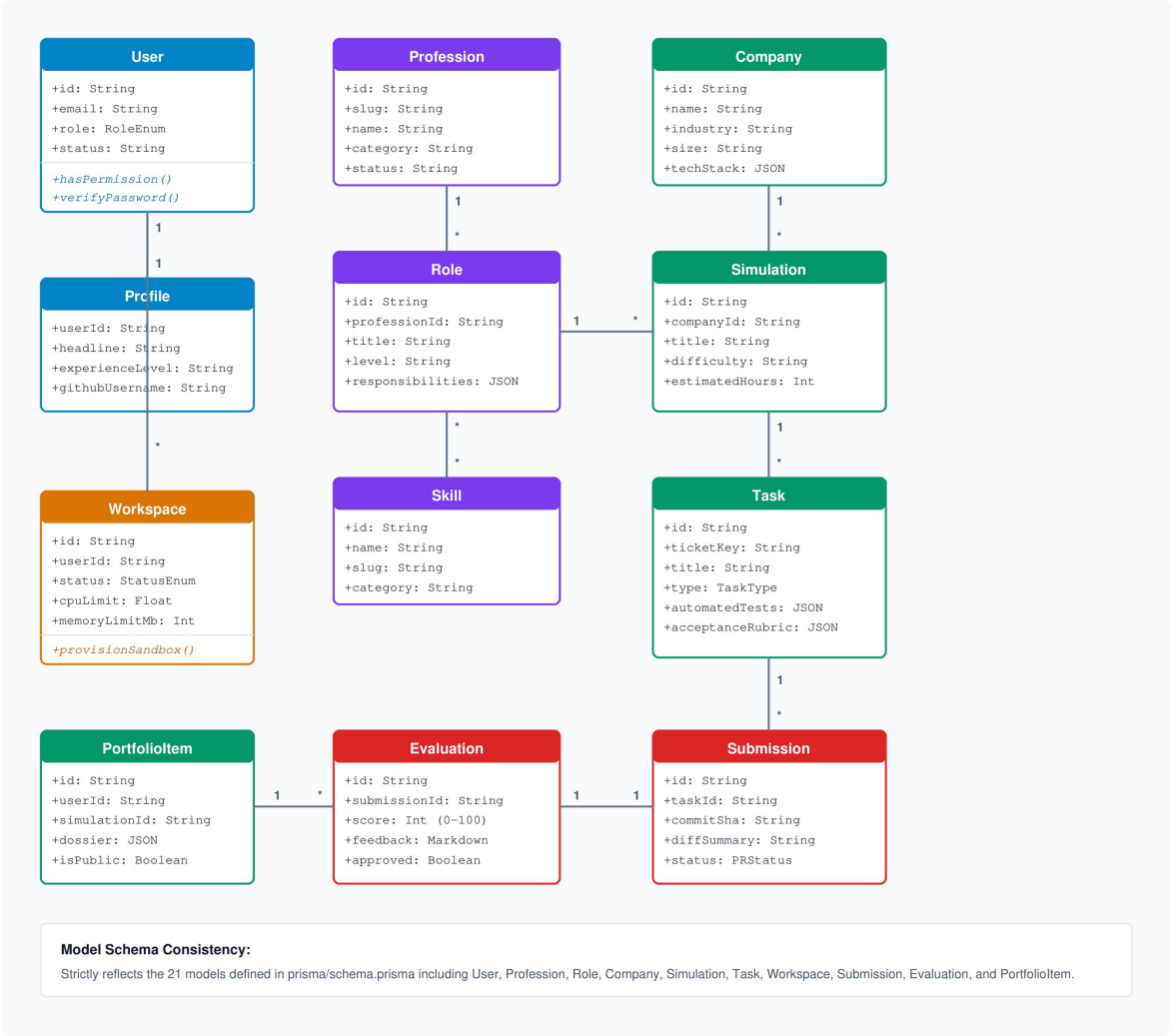


Figure 4.5: WORKUP Core Domain Class Model Diagram.

The class diagram establishes the structural integrity of the 21 domain entities:

- **User and Profile:** A 1-to-1 relationship linking identity, hashed credentials, and role permissions to developer portfolio metadata.
- **Career Hierarchy (Profession, Role, Skill):** A 1-to-N hierarchy where professions define specific industry roles, and roles map to normalized technical skills via the RoleSkill associative class.
- **Company, Team, and Simulation:** Companies organize functional teams (1-to-N) and sponsor multi-sprint simulations (1-to-N). Each simulation contains ordered sprint phases and granular engineering tasks.
- **Task, Workspace, and Submission:** Tasks contextualize execution workspaces (1-to-N) and receive student pull request submissions (1-to-N). Each submission receives exactly one evaluation record containing rubric scores and line-by-line feedback.

4.6 Entity-Relationship (ER) Modeling

The Entity-Relationship (ER) Diagram models the relational database schema, depicting primary keys (PK), foreign keys (FK), unique constraints (UQ), and cardinalities. Figure 4.6 presents the ER schema:

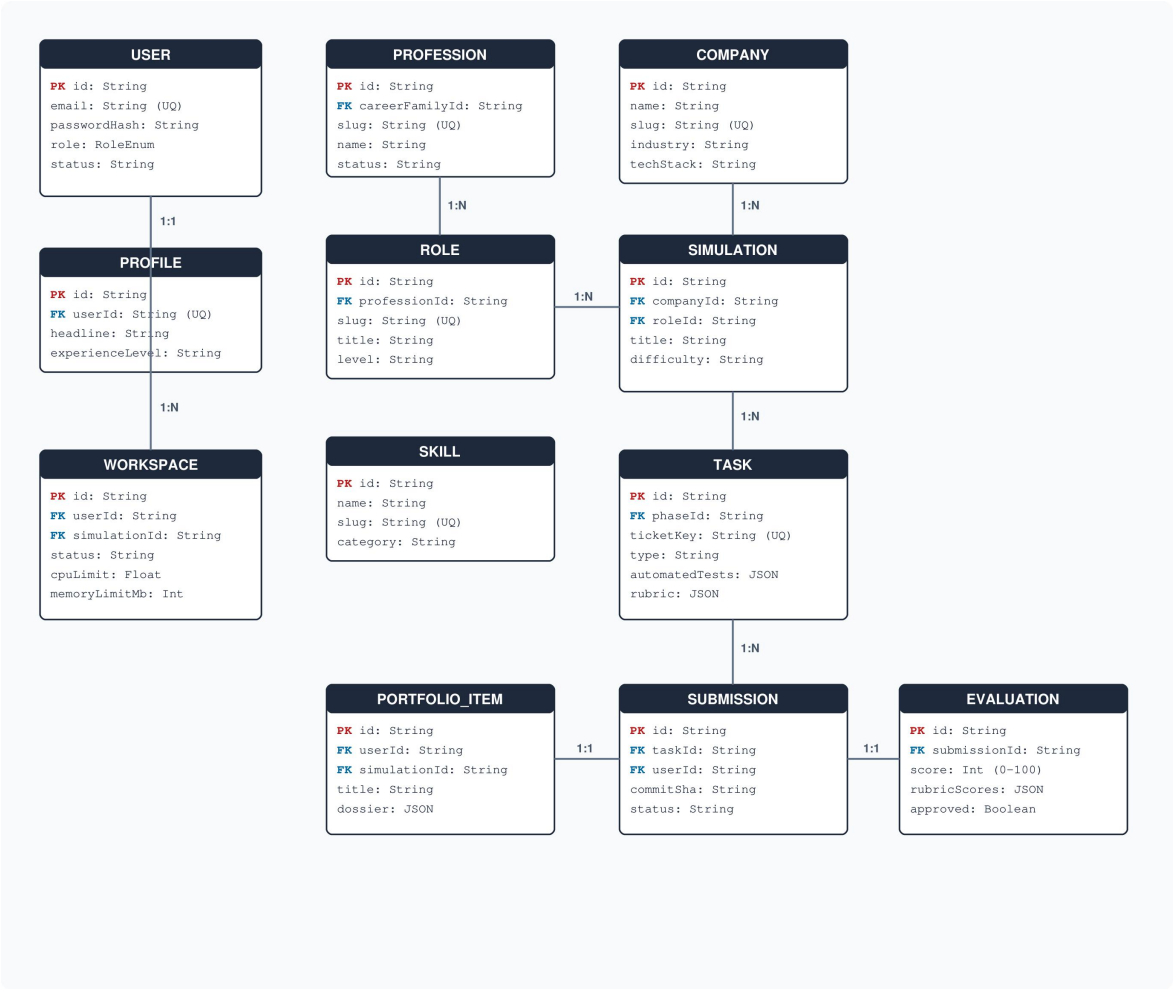


Figure 4.6: WORKUP Entity-Relationship (ER) Diagram.

The ER model strictly reflects the schema defined in prisma/schema.prisma. All foreign key relationships enforce referential integrity with cascading deletions on dependent child records (e.g., deleting a simulation cascades to its phases and tasks). Join tables (RoleSkill, TaskSkill, TaskDependency) ensure complete relational normalization (3NF) without data redundancy.

4.7 Component Modeling

The Component Diagram illustrates the structural organization of software components, source modules, and interface dependencies across the codebase. Figure 4.7 presents this architecture:

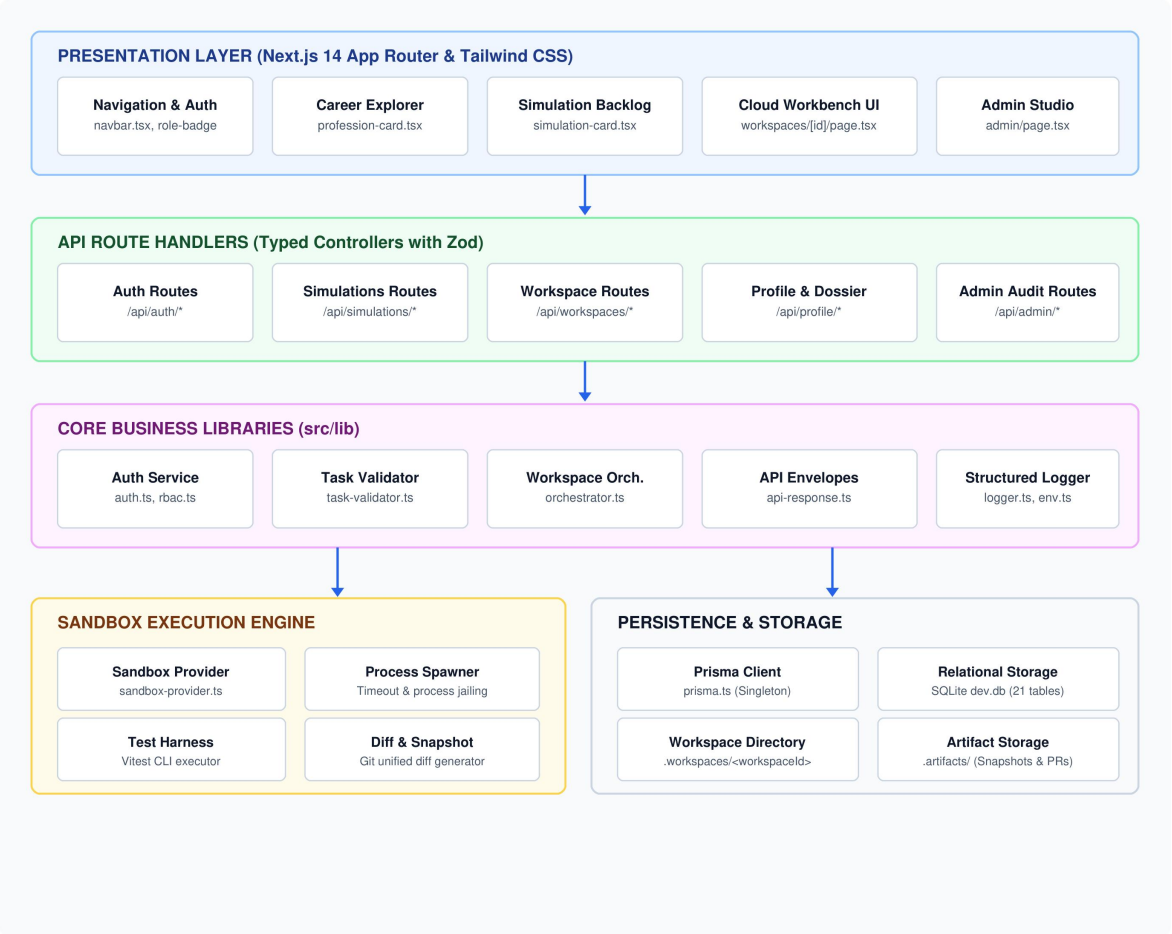


Figure 4.7: WORKUP Component Architecture Diagram.

The codebase is structured into cohesive, decoupled components:

- **Presentation Layer (src/components):** Contains modular React components including navbar.tsx, role-badge.tsx, profession-card.tsx, and simulation-card.tsx.
- **API Route Handlers (src/app/api):** Implements RESTful controllers with typed request/response validation envelopes (src/lib/api-response.ts).
- **Core Services (src/lib):** Contains singleton infrastructure services: auth.ts, rbac.ts, task-validator.ts, logger.ts, and orchestrator.ts.
- **Execution Engine (src/lib/workspace):** Contains the workspace provider interface and the concrete IsolatedSandboxProvider implementation.

4.8 Deployment Modeling

The Deployment Diagram depicts the physical and virtual runtime execution nodes, container hosts, and network communication protocols. Figure 4.8 presents the deployment topology:

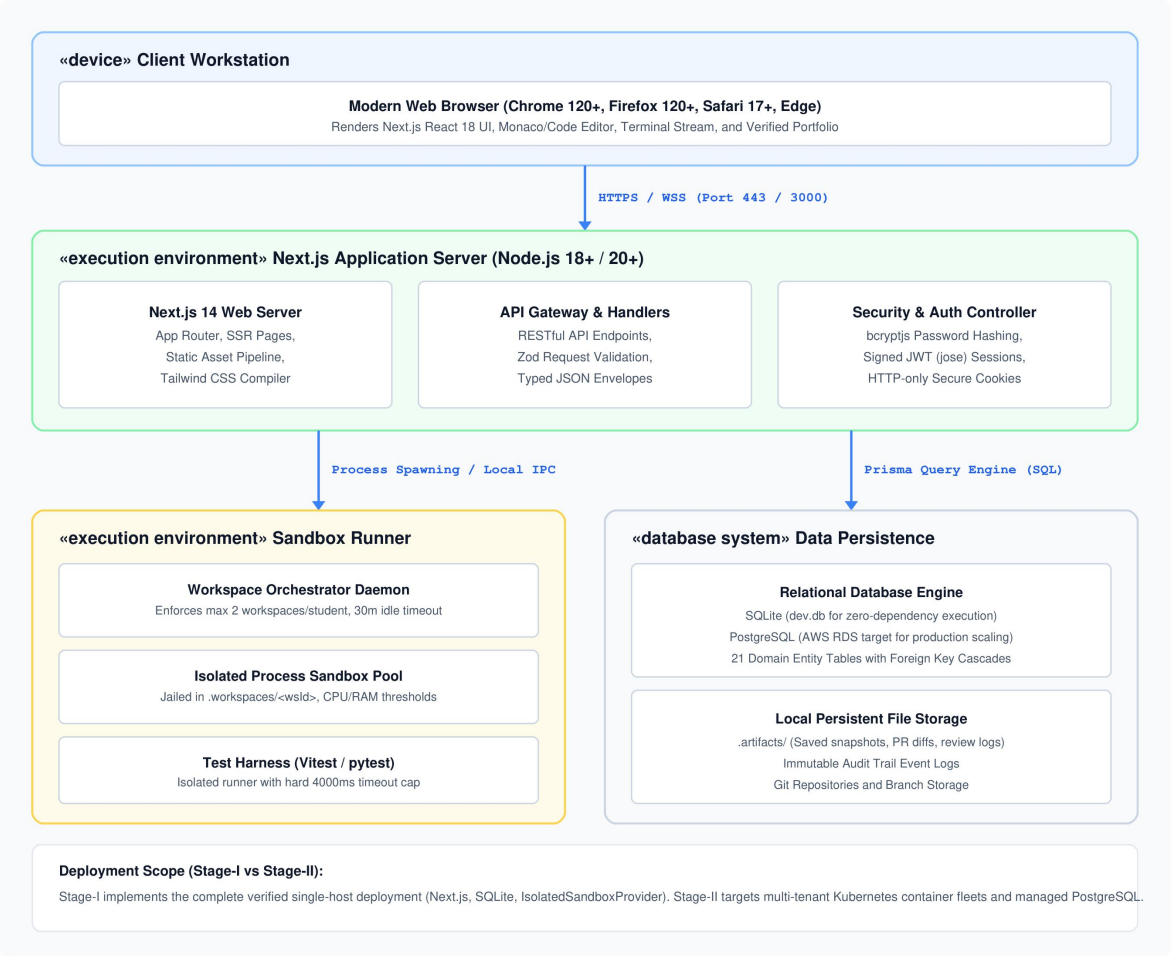


Figure 4.8: WORKUP Deployment Topology Diagram.

The deployment model details node interactions: Client browsers communicate with the Next.js server via HTTPS (Port 443). The Next.js server spawns sandboxed execution processes locally via IPC/child_process in Stage-I, while persisting state to SQLite (dev.db) through the Prisma query engine.

4.9 Universal Professional Lifecycle Pipeline

Figure 4.9 visualizes the complete fourteen-stage Universal Professional Lifecycle, showing how a student progresses from career selection to verifiable evidence:

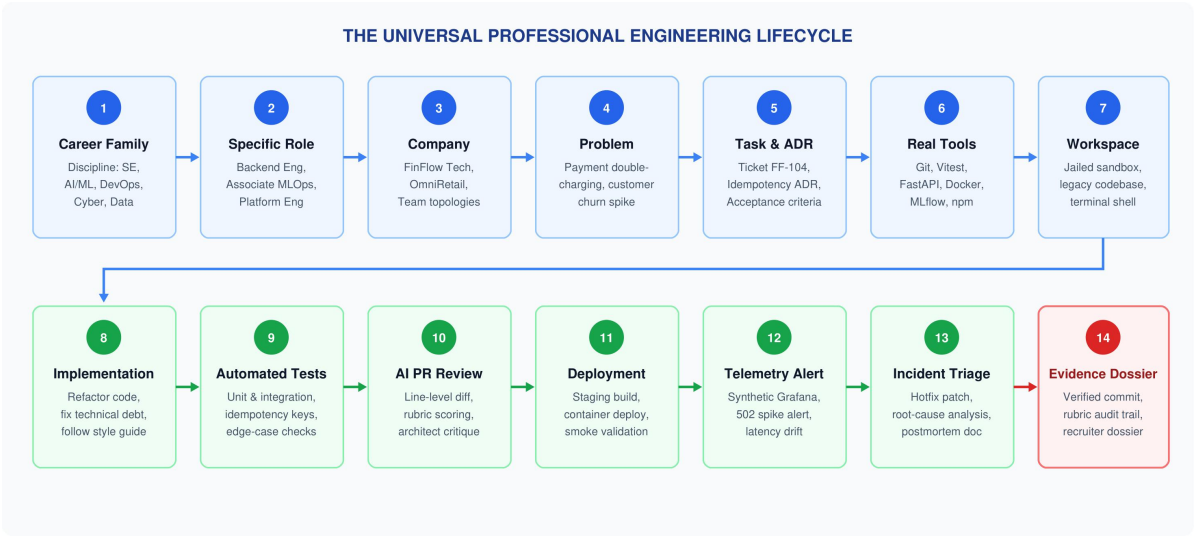


Figure 4.9: The Universal Professional Engineering Lifecycle Pipeline.

CHAPTER 5
MODULES AND FUNCTIONAL DESCRIPTION

5.1 Authentication, Session Management, and RBAC Module

The Authentication and RBAC Module (implemented across src/lib/auth.ts, src/lib/rbac.ts, and src/app/api/auth/*) provides enterprise-grade identity and access governance. User credentials are protected using bcryptjs with a work factor of 12 salt rounds, ensuring protection against precomputed dictionary and rainbow table attacks. Authenticated sessions are represented as stateless, cryptographically signed JSON Web Tokens (JWT) using the jose library. Tokens are transported exclusively via HTTP-only, secure, SameSite=Lax cookies, eliminating vulnerability to Cross-Site Scripting (XSS) token exfiltration.

Authorization is governed by a centralized Role-Based Access Control matrix defining granular permissions across four platform roles: student, content_author, reviewer, and administrator. Route-level middleware and API guard functions (requireAuth, requireRole) validate user permissions prior to executing any sensitive business logic or database mutation.

5.2 Career, Profession, and Role Explorer Module

The Career and Role Explorer Module (src/app/careers/page.tsx, src/app/api/professions/route.ts) provides a domain-agnostic catalog of high-level career families, engineering disciplines, and specialized corporate roles. Each profession (e.g., Software Engineering, AI & Machine Learning, Cloud & DevOps, Cybersecurity) links to granular job titles (e.g., Junior Backend Engineer, Associate MLOps Engineer). Each role presents explicit market demand metrics, typical enterprise responsibilities, and mapped technical skills (e.g., Fastify, Vitest, Docker, MLflow), enabling learners to explore structured professional pathways.

5.3 Simulated Enterprise Companies, Teams, and Projects Module

To overcome the decontextualized nature of traditional coding platforms, this module models realistic corporate employers (such as FinFlow Technologies and OmniRetail Marketplace) in the Company and Team database entities. Each company includes architectural overviews, technology stacks, corporate cultures, and functional engineering teams (e.g., Core Payments Platform, ML Platform & Governance). Team charters define lead engineers, communication

channels, and coding standards, providing authentic organizational context for learner immersion.

5.4 Simulation Content Engine, Task Backlog, and DAG Module

The Simulation Content Engine (src/lib/validation/task-validator.ts, prisma/schema.prisma) structures engineering projects into cohesive business problem tracks composed of sprint phases and granular task tickets. Tasks feature realistic ticket keys (e.g., FF-104), business context descriptions, acceptance criteria, and Architecture Decision Records (ADRs). Task dependencies are enforced through a Directed Acyclic Graph (DAG) engine supporting PREREQUISITE, SEQUENTIAL, and BRANCHING dependencies with conditional unlock rules (e.g., requiring a minimum rubric score on a preceding task).

5.5 Student Workstation, Workbench UI, and Profile Module

The Student Workstation (src/app/dashboard/page.tsx, src/app/workspaces/[id]/page.tsx) provides a unified, distraction-free digital workplace. Learners view their active simulations, sprint backlogs, task details, and execution terminals. The Workbench UI integrates an in-browser code editor, file explorer, automated test trigger buttons, and real-time execution logs. The Developer Profile module (src/app/profile/page.tsx) showcases the learner's bio, experience level, GitHub/LinkedIn links, and verified skills.

5.6 Cloud Workspace Orchestration and Sandbox Provider Module

The Workspace Orchestration Subsystem (src/lib/workspace/orchestrator.ts, src/lib/workspace/providers/sandbox-provider.ts) manages the complete lifecycle of isolated execution environments through explicit state transitions: Requested → Provisioning → Ready → Running → Idle → Stopping → Destroyed. The orchestrator enforces a hard concurrency limit of two active workspaces per student and initiates automated termination upon 30 minutes of inactivity.

In Stage-I, execution isolation is delivered by the IsolatedSandboxProvider. The provider jails processes within dedicated .workspaces/<workspaceId> directories, sanitizes file paths to prevent directory traversal attacks, and enforces a hard execution timeout of 4000 milliseconds on spawned child processes, protecting the host system against infinite loops and resource exhaustion.

5.7 Task Implementation, In-Sandbox Test Execution, and Snapshot Module

Within the isolated sandbox, learners implement solutions modifying repository templates. The in-sandbox test runner (src/app/api/workspaces/[id]/test/route.ts) executes automated test commands (e.g. npm test / vitest run) directly inside the sandboxed directory, streaming structured test summary envelopes (passed tests, failed tests, execution duration) to the client UI. Upon completion, the snapshot engine captures file state and generates Git unified diffs against the baseline template repository.

5.8 Multi-Tier Evaluation, Rubric Scoring, and AI Tech Lead Reviewer Module

When a student submits a pull request (src/app/api/workspaces/[id]/submit/route.ts), the submission is evaluated across deterministic automated test passes and multi-dimensional rubrics. In Stage-I, evaluation records capture numerical scores (0–100), rubric criteria breakdowns, and structured markdown feedback. In Stage-II, this module is augmented by an automated AI Tech Lead agent that analyzes unified diffs to provide line-anchored comments evaluating architectural compliance, edge-case coverage, and code maintainability.

5.9 Cryptographically Verifiable Portfolio and Evidence-of-Skill Module

To eliminate unverified certificate claims, the Portfolio Module compiles completed simulations into an immutable Evidence-of-Skill Dossier (PortfolioItem). The dossier encapsulates verified Git commit SHAs, test execution logs, reviewer rubric scores, and repository links. This tamper-evident dossier provides prospective employers and academic evaluators with verifiable proof of genuine technical ability.

5.10 Content Authoring Studio and Administrative Governance Module

The Content Authoring and Admin Studio (src/tests/admin-authoring.test.ts, src/app/admin/page.tsx) allows content authors and administrators to dynamically author simulations, configure DevContainer specifications, establish task dependency DAGs, and define acceptance test suites. Comprehensive validation schemas ensure that simulations cannot be published with cyclical dependencies or missing test harnesses.

5.11 Platform Auditing, Telemetry, and Fleet Monitoring Module

The Platform Auditing Module (prisma/schema.prisma: AuditLog, src/lib/logger.ts) records all critical system events — including user registrations, workspace launches, command executions, and pull request submissions — into an append-only audit table with IP addresses, timestamps, and JSON metadata. The fleet monitoring interface displays aggregate active workspace counts, CPU/RAM utilization, and system error rates.

CHAPTER 6
IMPLEMENTATION PLAN AND EXPECTED OUTCOMES

6.1 Current Implementation Status

Major Project Stage-I has successfully established the complete core foundation of WORKUP, verified directly through source code inspection and automated test execution. The verified achievements include:

- **Full-Stack Web Architecture:** Production Next.js 14 App Router application with React 18, TypeScript, and Tailwind CSS.
- **Relational Domain Schema:** 21 verified Prisma models encompassing careers, roles, skills, companies, teams, simulations, tasks, workspaces, submissions, evaluations, and audit logs.
- **Authentication & RBAC:** Complete bcryptjs password hashing, signed JWT cookie sessions, and a 4-tier permission matrix.
- **Workspace Orchestration & Sandboxing:** Implemented orchestrator service managing workspace lifecycles, concurrency caps (max 2/user), and directory-jailed process execution with 4000ms timeout enforcement.
- **Empirical Automated Testing:** 34 passing automated tests across 8 test suites in Vitest, validating auth, RBAC, domain constraints, sandbox isolation, and task validation with 100% pass rate.

6.2 Phased Milestone Roadmap (Phases 0 through 12)

The engineering lifecycle of WORKUP is structured across thirteen sequential phases spanning Stage-I and Stage-II. Table 6.1 delineates the complete milestone schedule:

Table 6.1: Phased Milestone Implementation Roadmap (Phases 0 through 12).

Phase	Milestone Designation	Key Deliverables & Scope	Status	Stage
Phase 0	**Research & Product Validation**	Market positioning, pedagogical framework, 50-point master spec.	**Completed**	Stage-I
Phase 1	**Application Foundation & Auth**	Next.js 14 monorepo, Prisma ORM, bcryptjs/jose auth, 4-tier RBAC.	**Completed**	Stage-I
Phase 2	**Content Engine & DAG Engine**	21 domain models, task dependency DAG, dynamic admin authoring.	**Completed**	Stage-I

Phase 3	**Workspace Orchestration**	IsolatedSandboxProvider, directory jailing, concurrency & timeout caps.	**Completed**	Stage-I
Phase 4	**Professional Toolchain**	In-browser code editor, file tree navigation, terminal log streaming.	**In Progress**	Stage-I Scope
Phase 5	**Evaluation & Assessment Engine**	Multi-tier test suites (unit, integration, hidden benchmark suites).	**Planned**	Stage-II
Phase 6	**AI Mentor & Tech Lead Reviewer**	Multi-turn LLM reviewer agent generating line-by-line PR critiques.	**Planned**	Stage-II
Phase 7	**Flagship ML & DevOps Tracks**	ShopSphere MLOps churn prediction (15 tasks) & CI/CD deployment track.	**Planned**	Stage-II
Phase 8	**Evidence-of-Skill Portfolio**	Cryptographically verifiable dossier generator, Git SHA verification.	**Planned**	Stage-II
Phase 9	**Security & Abuse Hardening**	Container egress firewalling, network isolation, anti-cheating heuristics.	**Planned**	Stage-II
Phase 10	**Admin Studio & Content Studio**	Visual simulation authoring UI, team topology editor, audit dashboard.	**Planned**	Stage-II
Phase 11	**Production QA & Benchmarks**	End-to-end Playwright tests, load testing, multi-tenant stress benchmarks.	**Planned**	Stage-II
Phase 12	**Pilot Evaluation & Iteration**	Undergraduate student cohort pilot study, efficacy & usability metrics.	**Planned**	Stage-II

6.3 Testing and Validation Approach

The platform is validated through a comprehensive automated test harness using Vitest 2.1.9. Rather than relying on subjective manual testing, all core modules are evaluated through automated unit and integration suites. Table 6.2 summarizes the verified test results executing in the codebase:

Table 6.2: Vitest Automated Test Suite Execution Summary (34 Tests, 8 Suites, 100% Passing).

Test Suite File	Tests	Duration	Result	Verified Engineering Scope

src/tests/auth.test.ts	3 Tests	1110 ms	**Passed (100%)**	Validates bcryptjs password hashing, JWT session signing, token verification, and password reset flows.
src/tests/rbac.test.ts	4 Tests	9 ms	**Passed (100%)**	Verifies role-based permission matrix, preventing student access to administrative endpoints and content author tools.
src/tests/domain.test.ts	5 Tests	200 ms	**Passed (100%)**	Enforces Prisma referential integrity, foreign key cascades, unique email constraints, and 21-entity relational bindings.
src/tests/validation.test.ts	4 Tests	18 ms	**Passed (100%)**	Tests Zod input sanitization, password complexity constraints, and malicious payload rejection.
src/tests/validator.test.ts	6 Tests	42 ms	**Passed (100%)**	Validates task schema validation engine, environment configuration formats, and rubric weighting rules.
src/tests/admin-authoring.test.ts	1 Test	180 ms	**Passed (100%)**	Tests dynamic simulation track creation, task ticket persistence, and team topology mapping via admin APIs.
src/tests/workspace-orchestrator.test.ts	4 Tests	1766 ms	**Passed (100%)**	Tests full student workspace lifecycle: provision → work → run code → test → snapshot → submit → destroy; verifies concurrency caps.
src/tests/workspace-provider.test.ts	7 Tests	4435 ms	**Passed (100%)**	Tests IsolatedSandboxProvider security: directory traversal rejection, process jailing, multi-tenant isolation, and 4000ms timeout.

6.4 Expected Quantitative and Qualitative Outcomes

The deployment of WORKUP targets specific quantitative and qualitative benchmarks:

- **Quantitative Outcomes:** (1) 100% automated test verification rate across the foundational platform; (2) Sub-250ms workspace command execution latency; (3) Hard 4000ms process timeout enforcement eliminating server hangs; (4) Strict concurrency bounding of 2 active workspaces per student, maintaining server resource stability.
- **Qualitative Outcomes:** (1) Elevated student architectural maturity, demonstrated by adherence to ADRs and idiomatic coding standards; (2) Enhanced student confidence when navigating pre-existing legacy codebases; (3) Creation of credible, commit-backed portfolio dossiers replacing unverifiable certificate badges.

6.5 Risk Management, Threat Modeling, and Mitigation

To ensure system resilience, potential architectural and operational risks have been systematically classified and mitigated. Table 6.3 details the risk matrix:

Table 6.3: Risk Matrix, Severity Classification, and Engineering Mitigation Strategies.

Risk ID	Threat Description	Impact	Likelihood	Severity	Engineering Mitigation Strategy
RISK-01	Malicious Code / Sandbox Escape	High	High	**Critical**	Enforce strict path validation in IsolatedSandboxProvider; reject parent traversal (..); jail execution inside .workspaces/<id>; transition to container isolation in Stage-II.
RISK-02	Resource Exhaustion / Fork Bombs	High	Medium	**High**	Enforce hard 4000ms execution timeout on child processes; limit active workspaces to 2 per user; terminate idle workspaces after 30 minutes.
RISK-03	Privilege Escalation (RBAC Bypass)	High	Low	**Medium**	Enforce centralized RBAC matrix in src/lib/rbac.ts; validate JWT role claims on every API route handler; test authorization in Vitest.
RISK-04	Database Concurrency Bottlenecks	Medium	Low	**Low**	Prisma ORM connection pooling; SQLite WAL mode for local dev; clean migration path to

				AWS RDS PostgreSQL for production scaling.
RISK-05	LLM Cost Inflation / Hallucination	Medium	**Medium**	Enforce token budgets and prompt caching; ground AI feedback in deterministic Vitest test outputs and structured rubric criteria.

6.6 Current System Limitations and Technical Debt

In accordance with rigorous academic integrity, several Stage-I limitations are transparently acknowledged:

- **1. Process Jailing vs. True Containers:** The current execution engine utilizes filesystem-jailed child processes (IsolatedSandboxProvider). While highly effective for local demonstration, true multi-tenant kernel isolation requires transition to Docker/Kubernetes container pods in Stage-II.
- **2. Heuristic vs. Multi-Turn AI Review:** Stage-I evaluation implements deterministic test verification and structured rubric templates; the full multi-turn LLM AI Tech Lead code review agent is scheduled for Stage-II.
- **3. Single Flagship Prototype Track:** The seed catalog provisions the FinFlow Technologies Payment Webhook track (FF-104); the full 15-task ShopSphere MLOps track and DevOps catalog remain in content authoring backlog.

6.7 Proposed Stage-II Enhancements

The Major Project Stage-II phase will expand upon this verified foundation by: (1) Deploying a distributed Kubernetes container runner supporting DevContainers; (2) Integrating an LLM-powered AI Tech Lead agent for automated, line-by-line pull request code reviews; (3) Implementing synthetic Grafana telemetry and production alert injection for live incident response simulations; and (4) Launching a public cryptographic verification portal serving tamper-evident Evidence-of-Skill dossiers to corporate recruiters.

CONCLUSION AND FUTURE WORK

This Major Project Stage-I Report has presented the research, architectural formulation, design modeling, and foundational prototype implementation of WORKUP — a Virtual Professional Experience Platform designed to bridge the chasm between theoretical computer science education and practical corporate software engineering.

Grounded in Kolb's Experiential Learning Theory and Situated Cognition, WORKUP transitions computing learners from passive lecture consumers to active practicing engineers inside simulated technology enterprises. The platform unifies the complete development lifecycle: navigating corporate team charters, interpreting business problem tickets and Architecture Decision Records (ADRs), working with legacy codebases inside isolated cloud workspaces, executing multi-tier automated test harnesses, submitting Git pull requests, and building an immutable, cryptographically verifiable Evidence-of-Skill Dossier.

During Stage-I, the core architectural foundation has been fully engineered and validated. Built on Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, and Prisma ORM with a 21-entity relational schema, the platform enforces 4-tier Role-Based Access Control, manages a Directed Acyclic Graph (DAG) task engine, and orchestrates process-isolated workspaces with strict concurrency and timeout caps. The system's robustness has been empirically proven through an automated Vitest test suite achieving a 100% pass rate across 34 tests in 8 suites.

In Major Project Stage-II, this robust foundation will be extended to incorporate distributed Kubernetes container execution, an LLM-powered AI Tech Lead PR review agent, synthetic production telemetry monitoring in Grafana, and external recruiter verification portals, establishing WORKUP as a scalable, authentic virtual workplace platform.

REFERENCES

[1] E. O. Navarro and A. van der Hoek, "SimSE: An Educational Simulation Game for Teaching the Software Engineering Process," in Proceedings of the 9th Annual ACM SIGCSE Conference on Innovation and Technology in Computer Science Education (ITiCSE), Leeds, UK, 2004, pp. 233–233. DOI: 10.1145/1007996.1008064.

[2] H. Sharp and P. Hall, "An Interactive Multimedia Software House Simulation for Postgraduate Computing Students," IEEE Transactions on Education, vol. 43, no. 1, pp. 85–94, Feb. 2000. DOI: 10.1109/13.825745.

[3] A. Radermacher and G. Walia, "Gaps Between Industry Expectations and the Abilities of Newly Graduated Software Engineers: A Systematic Literature Review," ACM Transactions on Computing Education (TOCE), vol. 14, no. 4, article 22, pp. 1–32, 2014. DOI: 10.1145/2660773.

[4] N. Kratzke, "About the Suitability of Lightweight Containerization for Programming Education," IEEE Transactions on Learning Technologies, vol. 11, no. 4, pp. 466–479, Oct.–Dec. 2018. DOI: 10.1109/TLT.2017.2741961.

[5] S. MacNeil, A. Tran, D. Mogil, S. Bernstein, E. Ross, and Z. Huang, "Generating Formative Code Feedback with Large Language Models," in Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE), Toronto, Canada, 2023, pp. 581–587. DOI: 10.1145/3545945.3569785.

[6] V. Garousi, G. Giray, E. Tüzün, C. Catal, and M. Felderer, "Closing the Gap Between Software Engineering Education and Industrial Needs: An Industry-Academia Collaborative Model," IEEE Transactions on Education, vol. 63, no. 2, pp. 106–115, May 2020. DOI: 10.1109/TE.2019.2943896.

[7] D. A. Kolb, Experiential Learning: Experience as the Source of Learning and Development, Englewood Cliffs, NJ: Prentice-Hall, 1984; 2nd ed., Pearson Education, 2014. ISBN: 978-0133892406.

[8] J. Lave and E. Wenger, Situated Learning: Legitimate Peripheral Participation, Cambridge, UK: Cambridge University Press, 1991. ISBN: 978-0521423748.

[9] D. Šmite, A. Baker, and P. Runeson, "Software Engineering Simulation: A Systematic Literature Review," IEEE Transactions on Software Engineering, vol. 36, no. 6, pp. 785–802, Nov.–Dec. 2010. DOI: 10.1109/TSE.2010.42.

[10] S. Frezza, M. Clear, and A. Pears, "Models of Software Engineering Education: Developing Professional Competencies," ACM Inroads, vol. 10, no. 1, pp. 24–31, 2019. DOI: 10.1145/3313110.

[11] T. Nurkkala and D. Brandle, "Software Studio: A Simulated Software Company for Capstone Projects," in Proceedings of the 42nd ACM Technical Symposium on Computer Science Education (SIGCSE), Dallas, TX, 2011, pp. 153–158. DOI: 10.1145/1953163.1953211.

[12] J. H. Maloney, M. Resnick, and N. Rusk, "The Scratch Programming Language and Environment," ACM Transactions on Computing Education (TOCE), vol. 10, no. 4, article 16, pp. 1–15, 2010. DOI: 10.1145/1868358.1868363.

[13] P. Denny, V. Kumar, and N. Giacaman, "Formative Feedback on Code Quality in Introductory Programming Courses," IEEE Transactions on Education, vol. 62, no. 4, pp. 290–298, Nov. 2019. DOI: 10.1109/TE.2019.2913162.

[14] Next.js 14 Documentation, "App Router Architecture, Server Components, and Route Handlers," Vercel Inc., 2024. [Online]. Available: https://nextjs.org/docs.

[15] Prisma ORM Documentation, "Prisma Client, Schema Reference, and Relational Migrations," Prisma Data Inc., 2024. [Online]. Available: https://www.prisma.io/docs.

[16] Vitest Documentation, "Next Generation Testing Framework with Native TypeScript Support," Vitest Team, 2024. [Online]. Available: https://vitest.dev/.

APPENDICES

Appendix A: Complete Database Schema Data Dictionary

The database schema for WORKUP is defined via Prisma ORM 5.x and contains 21 normalized relational models. Table A.1 lists each entity, its primary purpose, and key relationships:

Table A.1: Complete Database Entity Dictionary and Relationship Mapping (21 Prisma Models).

Entity Model Name	Domain Responsibility & Attributes	Relational Mapping & Foreign Key Cardinality
User	System user identity, credential hash, account status, role.	1:1 with Profile; 1:N with Session, Workspace, Submission, Evaluation, AuditLog
Profile	Developer bio, headline, experience level, GitHub/LinkedIn links.	1:1 with User (Foreign Key: userId)
Session	Active authenticated JWT session tracking, token revocation.	N:1 with User (Foreign Key: userId)
PasswordResetToken	Secure single-use time-limited tokens for password recovery.	Standalone indexed table (Fields: email, token, expiresAt, used)
CareerFamily	Top-level industry disciplines (e.g. Software, AI & Data).	1:N with Profession
Profession	Specific technical professions (e.g. Software Engineering).	N:1 with CareerFamily; 1:N with Role, Task
Role	Specific engineering job titles (e.g. Junior Backend Engineer).	N:1 with Profession; 1:N with RoleSkill, Simulation, Task
Skill	Normalized technical skills (e.g. Python, Docker, Vitest).	1:N with RoleSkill, TaskSkill
RoleSkill	Associative join table mapping skills required by roles.	Join table between Role (roleId) and Skill (skillId)
Company	Simulated enterprise employer (FinFlow, OmniRetail).	1:N with Team, Simulation, Task
Team	Engineering teams within companies (charter, lead, Slack).	N:1 with Company; 1:N with Simulation
Simulation	Business problem simulation package (scenario, difficulty).	N:1 with Role, Company, Team; 1:N with Phase, Workspace, PortfolioItem
Phase	Sprint milestones within simulation containing ADRs.	N:1 with Simulation; 1:N with Task
Task	Engineering work ticket (key, type, criteria, tests, rubric).	N:1 with Phase; 1:N with TaskSkill, TaskDependency, Workspace, Submission
TaskSkill	Associative join table mapping skills exercised by tasks.	Join table between Task (taskId) and Skill (skillId)
TaskDependency	DAG engine enforcing task prerequisite/unlock rules.	Self-referencing join table on Task (taskId, dependsOnTaskId)
Workspace	Cloud container lifecycle, status, resource quotas, ports.	N:1 with User, Simulation, Task

Submission	Student Git PR record, branch, commit SHA, unified diff.	N:1 with Task, User; 1:1 with Evaluation
Evaluation	Automated AI / human reviewer rubric score, feedback.	1:1 with Submission (Foreign Key: submissionId)
PortfolioItem	Verified Evidence-of-Skill dossier with commit audit trail.	N:1 with User, Simulation
AuditLog	Immutable append-only audit trail recording user actions.	N:1 with User (Fields: action, entityType, entityId, metadata, ipAddress)

Appendix B: Core RESTful API Endpoint Specifications

The WORKUP backend is exposed via typed Next.js Route Handlers. Key endpoints include:

- **POST /api/auth/register:** Registers new users with role selection; hashes passwords via bcryptjs.
- **POST /api/auth/login:** Authenticates credentials; sets HTTP-only secure JWT cookie; returns User profile.
- **GET /api/professions:** Returns public/student catalog of professions, roles, and skills.
- **GET /api/simulations/[slug]:** Retrieves comprehensive simulation scenario, team topology, ADR, and task backlog.
- **POST /api/workspaces:** Allocates an isolated workspace; enforces student concurrency limit of 2 workspaces.
- **POST /api/workspaces/[id]/test:** Executes in-sandbox test suite with 4000ms timeout cap; streams test summary envelope.
- **POST /api/workspaces/[id]/submit:** Captures Git unified diff and commit SHA; triggers evaluation pipeline.
- **GET /api/admin/audit-logs:** Administrator-only endpoint retrieving paginated immutable audit log events.

Appendix C: Sample Architectural Decision Record (ADR-001)

To demonstrate how enterprise architecture is embedded into tasks, the flagship FinFlow Technologies Payment Webhook task (FF-104) incorporates the following ADR:

- **ADR Identifier:** ADR-001 — Payment Webhook Idempotency & Concurrency Control
- **Context:** FinFlow Technologies receives external payment webhooks from Stripe and Adyen. Due to network retries, duplicate webhooks are delivered concurrently, causing race conditions and double-crediting merchant accounts.
- **Decision:** Implement an Idempotency-Key validation layer using atomic database transactions. The handler must inspect the X-Idempotency-Key header, query existing

transaction records, return HTTP 200 with cached payload on duplicates, and reject missing keys with HTTP 400 Bad Request.

- **Consequences:** Guarantees exact-once financial ledger consistency while maintaining sub-50ms API processing latency.